МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
«ЮЖНЫЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт компьютерных технологий и информационной безопасности

ОТЧЕТ

Практической работы №5
по теме: «Интеграции»
по дисциплине: «ВПК-8. Профессиональная разработка на платформе 1С:Предприятие»

Выполнил студент
группы КТбо4-7
Кострюкова К. А.

_____

Проверил
Ассистент кафедры САиТ
Борисов А. Ю.

Таганрог 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 Задание 4

2 Ход работы 6

2.1 Подготовка рабочего места 6

2.2 Задача 1. Выгрузка приходных накладных в JSON 8

2.3 Задача 2. Создание расходных накладных из JSON 12

2.4 Задача 3. Синхронизация баз «Головной офис» и «Филиал» 20

2.5 Задача 4. Загрузка контрагентов из Excel 31

2.6 Задача 5*. HTTP-запрос GET и заглушка Mockoon 35

2.7 Особенности реализации 40

ЗАКЛЮЧЕНИЕ 42

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 43

ПРИЛОЖЕНИЕ А (обязательное) Модуль объекта обработки «ОбменФилиалГоловнойОфис» 44

ПРИЛОЖЕНИЕ Б (обязательное) Модуль формы обработки «ИмпортРасходныхНакладных» 58

ВВЕДЕНИЕ

Интеграция — это обмен данными между программой «1С:Предприятие» и внешним миром: файлами, другими информационными базами и веб-ресурсами. Для этого в платформе есть встроенные средства: чтение и запись JSON (объекты ЧтениеJSON, ЗаписьJSON и сериализатор XDTO), табличный документ, который умеет читать файлы Excel, и объекты HTTPСоединение и HTTPЗапрос для обращения к HTTP-ресурсам.

Цель работы — научиться разрабатывать интеграции в конфигурации «1С:Управление нашей фирмой» (УНФ) и проверять обмен данными.

Для достижения цели решены следующие задачи:

– разработана обработка выгрузки приходных накладных за период в файл JSON с вложенным массивом товаров;

– разработана обработка, которая создает расходные накладные по этому файлу, сопоставляя справочники по наименованию;

– организована автоматическая синхронизация двух баз УНФ — «Головной офис» и «Филиал»;

– разработана обработка загрузки контрагентов из файла Excel с контролем дублей по ИНН;

– разработана обработка, выполняющая HTTP-запрос GET; она проверена на заглушке, поднятой в программе Mockoon.

Работа выполнена в демонстрационной базе конфигурации УНФ редакции 3.0 (учебная, версия 3.0.13.251) на платформе «1С:Предприятие 8.5» (учебная версия 8.5.1.1150). Все решения сделаны как внешние обработки (файлы .epf), поэтому конфигурация баз не изменялась. Порядок работы основан на методических указаниях к практической работе [1], документации платформы [2], библиотеки стандартных подсистем [3] и программы Mockoon [4].

1 Задание

Все задачи выполняются в демонстрационной версии конфигурации «1С:Управление нашей фирмой».

Задача 1. Разработать обработку, которая выгрузит информацию из документов «Приходная накладная» за период, указанный на форме, во внешний файл в формате JSON. Табличную часть «Товары» необходимо выгрузить в отдельный массив объектов, строго привязанный к своему родительскому документу. Обязательны к выгрузке: дата документа, организация, контрагент, склад, а также номенклатура, количество и сумма по каждой строке товаров.

Задача 2. Разработать обработку, которая создаст новые документы «Расходная накладная» по JSON-файлу из задачи 1. Файл загружается в обработку по кнопке, после чего по кнопке «Создать» в базе создаются новые документы. Платформа должна присвоить им новые уникальные номера. Организации, контрагенты, склады и номенклатура сопоставляются с элементами текущей базы по наименованию. Документы нужно только записать в базу, без проведения.

Задача 3. Синхронизировать две конфигурации УНФ любым удобным способом. Одна конфигурация выполняет роль «Головного офиса», вторая — «Филиала». Документы, созданные в «Филиале», должны автоматически загружаться в «Головной офис», а новые элементы номенклатуры из «Головного офиса» — автоматически загружаться в «Филиал».

Задача 4. Создать обработку, в которую загружается файл Excel с контрагентами. По кнопке «Загрузить» должны появиться новые элементы справочника; если такой контрагент уже есть, пользователь получает сообщение. Дубли проверяются по ИНН.

Задача 5*. Создать обработку, на форме которой можно ввести полный URL-адрес интернет-ресурса. По кнопке «GET» программа выполняет прямой GET-запрос на этот адрес и выводит полученный ответ. Для имитации внешнего HTTP-ресурса используется программа Mockoon: в ней

поднимается локальная заглушка (mock-сервер), которая возвращает ответ на запрос из 1С.

Результаты работы — пять внешних обработок (таблица 1) и настроенный обмен между двумя базами.

Таблица 1 — Задачи и файлы внешних обработок

Задача	**Файл**	**Где проверена**
1	ЭкспортНакладных.epf	Головной офис
2	ИмпортРасходныхНакладных.epf	Головной офис и Филиал
3	ОбменФилиалГоловнойОфис.epf	обе базы, подключена как дополнительная обработка
4	ИмпортКонтрагентов.epf	Головной офис
5*	HTTPЗапросGET.epf и окружение Mockoon Test.json	Головной офис

2 Ход работы

2.1 Подготовка рабочего места

Работа выполнялась в демонстрационной базе УНФ, развернутой в практической работе №4 из файла выгрузки «УНФ_Демо_Учебная.dt» (каталог C:\Users\krist\1C\УНФ_Демо). Для задачи 3 понадобилась вторая база, ее подготовка описана в подразделе 2.4. Учебная версия платформы разрешает только одно подключение к базе, поэтому для каждой базы запускался один клиент, а внешние обработки открывались в нем же.

Проверка имен метаданных. В конфигураторе («Конфигурация → Открыть конфигурацию») проверены точные имена объектов, с которыми работают обработки (таблица 2). Имена в УНФ отличаются от примера из методических указаний: табличная часть товаров называется «Запасы», склад хранится в реквизите «СтруктурнаяЕдиница», а поставщик и покупатель — в реквизите «Контрагент». Ключи JSON при этом остаются универсальными («Товары», «Склад», «Контрагент»), как требует задание.

Таблица 2 — Объекты метаданных, используемые в работе

Объект	Реквизиты и табличные части
Документ.ПриходнаяНакладная	Дата, Организация, Контрагент, СтруктурнаяЕдиница; табличная часть Запасы: Номенклатура, Количество, Сумма
Документ.РасходнаяНакладная	Дата, ВидОперации, Организация, Контрагент, Договор, СтруктурнаяЕдиница, СуммаДокумента; табличная часть Запасы: Номенклатура, ЕдиницаИзмерения, Количество, Цена, Сумма, СтавкаНДС, СуммаНДС, Всего
Справочник.Организации	Наименование
Справочник.Контрагенты	Наименование, НаименованиеПолное, ВидКонтрагента, ЮридическоеФизическоеЛицо, ИНН, КПП
Справочник.СтруктурныеЕдиницы	Наименование (склады и магазины)
Справочник.Номенклатура	Наименование, ЕдиницаИзмерения, ВидСтавкиНДС

Создание внешней обработки. Все обработки создаются по одной схеме:

1. В конфигураторе выбрать «Файл → Новый → Внешняя обработка», задать имя (например, «ЭкспортНакладных») и синоним (рисунок 1).

2. В ветке «Формы» нажать «Добавить» и создать управляемую форму обработки. Она становится основной формой — поле «Форма обработки» на вкладке «Основные».

3. В редакторе формы на вкладке «Реквизиты» добавить реквизиты формы с нужными типами и перетащить их на вкладку «Элементы», чтобы на форме появились поля ввода. На вкладке «Команды» создать команды и перетащить их на командную панель или в группу формы — появятся кнопки. Двойной щелчок по свойству «Действие» команды создает обработчик в модуле формы (вкладка «Модуль») (рисунок 2).

4. Сохранить обработку командой «Файл → Сохранить» в файл с расширением .epf.

5. Для проверки запустить базу в режиме «1С:Предприятие», открыть главное меню (значок в правой части заголовка окна), выбрать «Файл → Открыть…» и указать файл .epf. На предупреждение безопасности ответить «Да» — оно появляется для любого внешнего файла.

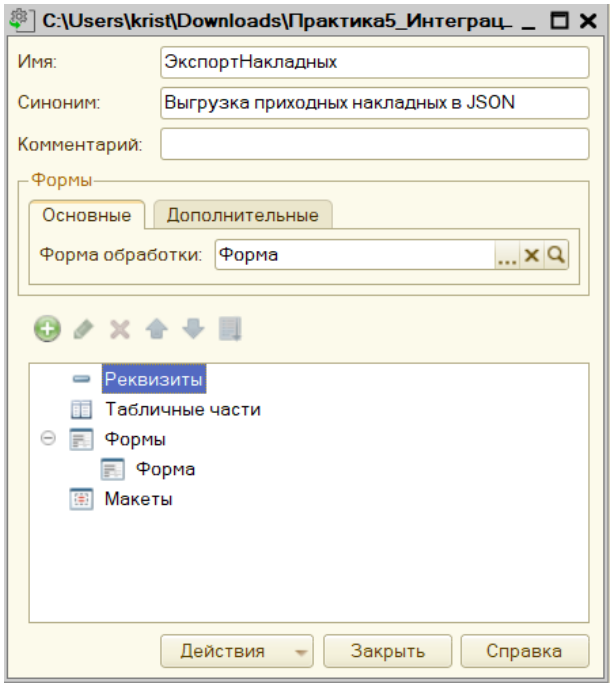


Рисунок 1 — Внешняя обработка «ЭкспортНакладных» в конфигураторе

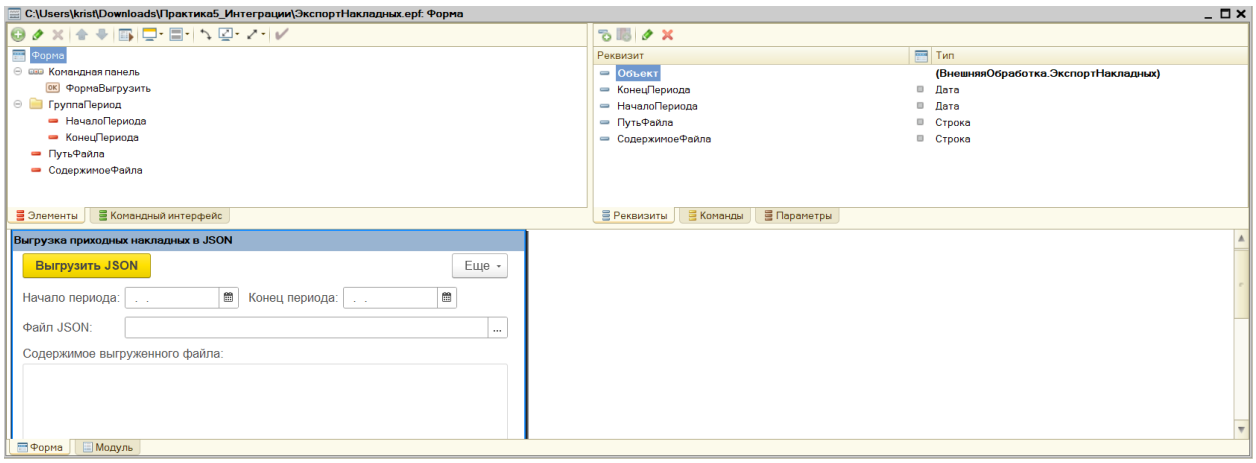


Рисунок 2 — Редактор формы обработки: элементы, реквизиты и вид формы

В модулях форм используются директивы компиляции: &НаКлиенте — для обработчиков кнопок и диалогов, &НаСервере — для работы с базой данных и файлами, &НаСервереБезКонтекста — для функций, которым не нужны данные формы. В УНФ отключены модальные окна (свойство конфигурации «Режим использования модальности» = «Не использовать»), поэтому диалоги выбора файла открываются асинхронно: методом Показать() с объектом ОписаниеОповещения, который указывает процедуру, вызываемую после выбора файла.

2.2 Задача 1. Выгрузка приходных накладных в JSON

Обработка «ЭкспортНакладных» (синоним «Выгрузка приходных накладных в JSON») выгружает приходные накладные за период в файл JSON.

Шаг 1. Форма. На форме созданы реквизиты из таблицы 3 и команда «Выгрузить» с заголовком «Выгрузить JSON», размещенная на командной панели формы как кнопка по умолчанию.

Таблица 3 — Реквизиты формы обработки «ЭкспортНакладных»

Реквизит	Тип	Назначение
НачалоПериода	Дата (состав «Дата»)	начало периода выгрузки
КонецПериода	Дата (состав «Дата»)	конец периода выгрузки
ПутьФайла	Строка (500)	путь к файлу JSON; кнопка выбора «…» открывает диалог сохранения
СодержимоеФайла	Строка (неогранич.)	текст выгруженного файла; многострочное поле только для просмотра

При открытии формы (обработчик ПриСозданииНаСервере) период заполняется текущим месяцем, а путь — значением C:\Exchange\incoming.json из методических указаний.

Шаг 2. Обработчик кнопки. Клиентская процедура Выгрузить проверяет, что заполнены период и путь и что начало периода не больше конца, затем вызывает серверную функцию ВыгрузитьНаСервере и выводит сообщение с количеством выгруженных накладных.

Шаг 3. Серверная функция (листинг 1) работает так:

– запрос выбирает ссылки на непомеченные на удаление приходные накладные с датой между началом первого и концом последнего дня периода (НачалоДня и КонецДня), упорядоченные по дате;

– для каждой накладной читается объект документа, и строки ее табличной части «Запасы» собираются в отдельный массив; этот массив кладется в структуру накладной под ключом «Товары». Поэтому товары каждого документа находятся внутри него и не перемешиваются с товарами других документов;

– дата записывается функцией XMLСтрока в формате ISO 8601 (например, 2024-04-24T13:40:25): его понимают другие системы, и в задаче 2 он читается обратно функцией XMLЗначение;

– ссылочные реквизиты (организация, контрагент, склад, номенклатура) выгружаются наименованиями — вспомогательной функцией ДокСтрокаИмя из методических указаний;

– массив накладных записывается в файл функцией ЗаписатьJSON в кодировке UTF-8 с отступами; если каталога файла нет, он создается;

– текст полученного файла выводится на форму, чтобы результат был виден сразу.

Листинг 1 — Серверная часть выгрузки приходных накладных

```
// Выгружает приходные накладные за период в JSON-файл.
// Каждая накладная - объект JSON, ее строки "Товары" - вложенный массив
внутри этого объекта,
// поэтому строки табличной части не перемешиваются между документами.
```

```
//
// Возвращаемое значение:
//  Число - количество выгруженных накладных.
//
&НаСервере
Функция ВыгрузитьНаСервере()

    Запрос = Новый Запрос;
    Запрос.Текст =
    "ВЫБРАТЬ
    |    Накладная.Ссылка КАК Ссылка
    |ИЗ
    |    Документ.ПриходнаяНакладная КАК Накладная
    |ГДЕ
    |    Накладная.Дата МЕЖДУ &Начало И &Конец
    |    И НЕ Накладная.ПометкаУдаления
    |
    |УПОРЯДОЧИТЬ ПО
    |    Накладная.Дата";
    Запрос.УстановитьПараметр("Начало", НачалоДня(НачалоПериода));
    Запрос.УстановитьПараметр("Конец", КонецДня(КонецПериода));
    Выборка = Запрос.Выполнить().Выбрать();

    РезультатJSON = Новый Массив;
    Пока Выборка.Следующий() Цикл

        Док = Выборка.Ссылка.ПолучитьОбъект();

        // В УНФ табличная часть с товарами называется "Запасы",
        // в файл она выгружается под универсальным ключом "Товары".
        СтрокиJSON = Новый Массив;
        Для Каждого СтрокаТовара Из Док.Запасы Цикл
            Позиция = Новый Структура;
            Позиция.Вставить("Номенклатура",
ДокСтрокаИмя(СтрокаТовара.Номенклатура));
            Позиция.Вставить("Количество", СтрокаТовара.Количество);
            Позиция.Вставить("Сумма", СтрокаТовара.Сумма);
            СтрокиJSON.Добавить(Позиция);
        КонецЦикла;

        Данные = Новый Структура;
        Данные.Вставить("ДатаДокумента", XMLСтрока(Док.Дата));
        Данные.Вставить("Организация", ДокСтрокаИмя(Док.Организация));
        Данные.Вставить("Контрагент", ДокСтрокаИмя(Док.Контрагент));
        Данные.Вставить("Склад", ДокСтрокаИмя(Док.СтруктурнаяЕдиница));
        Данные.Вставить("Товары", СтрокиJSON);
        РезультатJSON.Добавить(Данные);

    КонецЦикла;

    ФайлВыгрузки = Новый Файл(ПутьФайла);
    КаталогВыгрузки = Новый Файл(ФайлВыгрузки.Путь);
    Если Не КаталогВыгрузки.Существует() Тогда
        СоздатьКаталог(ФайлВыгрузки.Путь);
    КонецЕсли;

    ПараметрыЗаписи = Новый ПараметрыЗаписиJSON(ПереносСтрокJSON.Авто,
Символы.Таб);
    Запись = Новый ЗаписьJSON;
    Запись.ОткрытьФайл(ПутьФайла, "UTF-8", Ложь, ПараметрыЗаписи);
    ЗаписатьJSON(Запись, РезультатJSON);
    Запись.Закрыть();

    // Показываем полученный файл на форме
```

```
    Чтение = Новый ЧтениеТекста(ПутьФайла, КодировкаТекста.UTF8);
    СодержимоеФайла = Чтение.Прочитать();
    Чтение.Закрыть();

    Возврат РезультатJSON.Количество();

КонецФункции

// Представление ссылочного реквизита: наименование элемента справочника
// или пустая строка, если значение не заполнено.
&НаСервереБезКонтекста
Функция ДокСтрокаИмя(Значение)

    Если Не ЗначениеЗаполнено(Значение) Тогда
        Возврат "";
    КонецЕсли;

    Возврат Значение.Наименование;

КонецФункции
```

Шаг 4. Проверка. Обработка открыта в базе «Головной офис», задан период с 24.04.2024 по 26.04.2024. После нажатия «Выгрузить JSON» выведено сообщение «Выгружено накладных: 2», а на форме показано содержимое файла (рисунок 3).

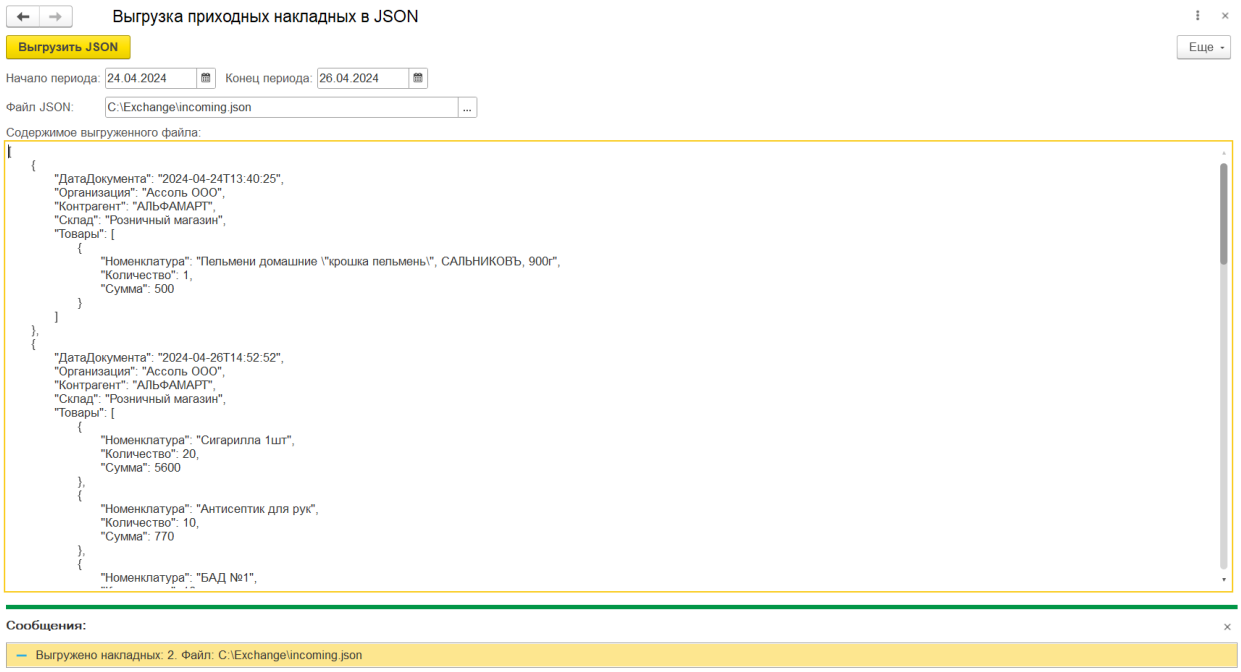


Рисунок 3 — Результат выгрузки приходных накладных в JSON

Структура полученного файла приведена в листинге 2: внешний массив объектов-накладных, у каждого свой массив «Товары». Первая накладная (24.04.2024) содержит одну строку на 500 руб., вторая (26.04.2024) — 21 строку на 879 828 руб.

Листинг 2 — Структура выгруженного файла C:\Exchange\incoming.json (вторая накладная сокращена)

```
[
    {
        "ДатаДокумента": "2024-04-24T13:40:25",
        "Организация": "Ассоль ООО",
        "Контрагент": "АЛЬФАМАРТ",
        "Склад": "Розничный магазин",
        "Товары": [
            {
                "Номенклатура": "Пельмени домашние \"крошка пельмень\",
САЛЬНИКОВЪ, 900г",
                "Количество": 1,
                "Сумма": 500
            }
        ]
    },
    {
        "ДатаДокумента": "2024-04-26T14:52:52",
        "Организация": "Ассоль ООО",
        "Контрагент": "АЛЬФАМАРТ",
        "Склад": "Розничный магазин",
        "Товары": [
            {
                "Номенклатура": "Сигарилла 1шт",
                "Количество": 20,
                "Сумма": 5600
            },
            ... еще 20 строк ...
        ]
    }
]
```

2.3 Задача 2. Создание расходных накладных из JSON

Обработка «ИмпортРасходныхНакладных» (синоним «Создание расходных накладных из JSON») читает файл из задачи 1 и создает по нему новые расходные накладные.

Шаг 1. Форма. Реквизиты формы приведены в таблице 4. Созданы две команды: «Загрузить» (заголовок «Загрузить JSON») и «Создать» (заголовок «Создать накладные»).

Таблица 4 — Реквизиты формы обработки «ИмпортРасходныхНакладных»

Реквизит	Тип	Назначение
ПутьJSON	Строка (500)	путь к файлу, кнопка выбора «…»
ЧислоЗаписей	Число (10)	количество накладных в файле, только просмотр
СписокНакладных	ТаблицаЗначений	накладные из файла: Дата, Организация, Контрагент, Склад, Позиций (заголовок «Строк»), Сумма, Результат
ТекстJSON	Строка (неогранич.)	текст загруженного файла; на форму не выводится
ТекстСообщения	Строка (неогранич.)	протокол, многострочное поле только для просмотра

Шаг 2. Загрузка файла. Кнопка «Загрузить JSON» открывает диалог выбора файла (рисунок 4). После выбора серверная процедура ЗагрузитьНаСервере читает текст файла (ЧтениеТекста, UTF-8) и передает его функции ПрочитатьНакладныеИзJSON, которая проверяет структуру:

– корень JSON — массив;

– каждый элемент массива — объект с ключами ДатаДокумента, Организация, Контрагент, Склад и Товары;

– значение ключа «Товары» — массив, а в каждой его строке есть ключи Номенклатура, Количество и Сумма.

При нарушении структуры вызывается исключение с понятным текстом, и пользователь видит сообщение «Файл не загружен: …». Если файл корректен, накладные показываются в таблице на форме, а текст файла сохраняется в реквизите ТекстJSON (рисунок 5).

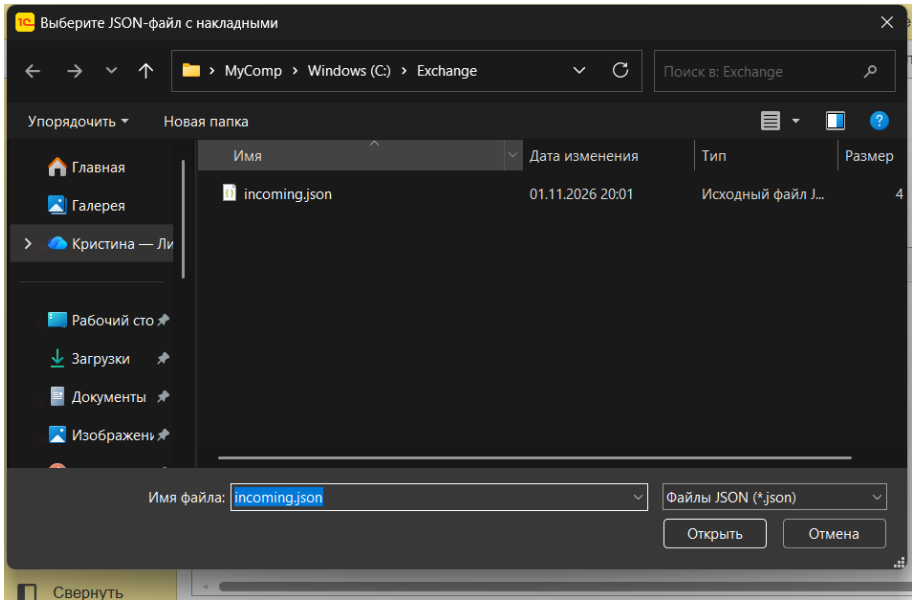


Рисунок 4 — Выбор JSON-файла в обработке создания расходных накладных

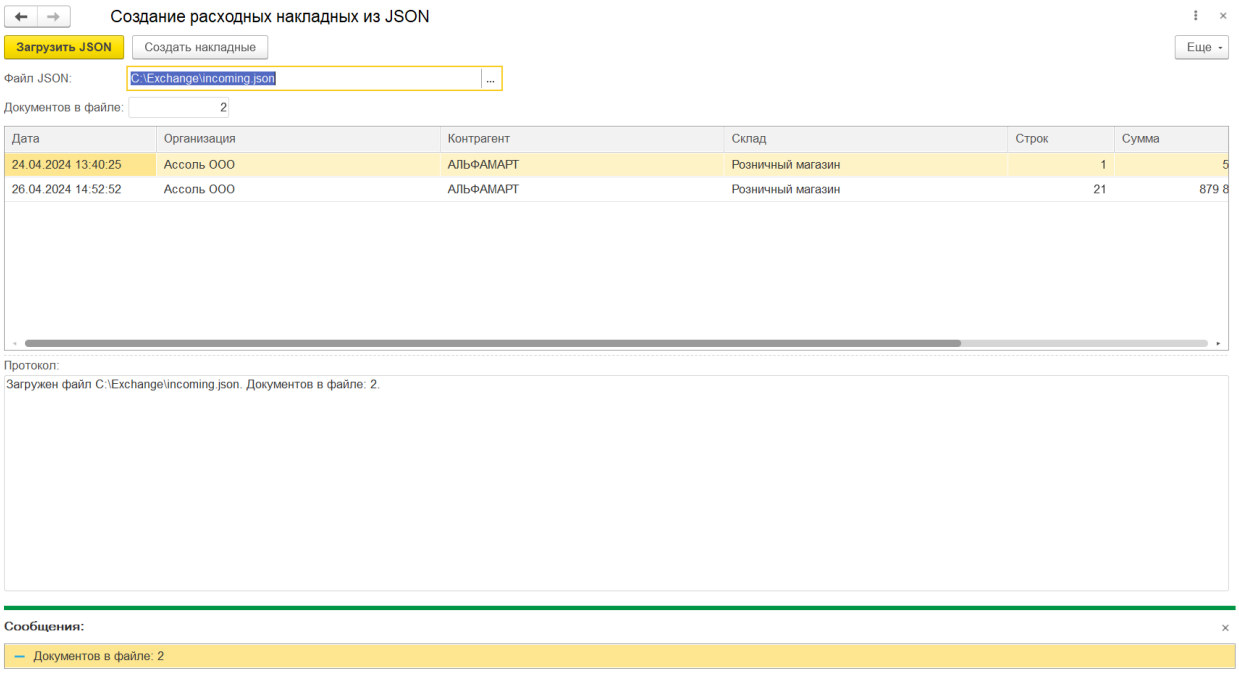


Рисунок 5 — Накладные, загруженные из JSON-файла

Шаг 3. Сопоставление по наименованиям. По кнопке «Создать накладные» серверная процедура СоздатьДокументыНаСервере сначала находит в базе все элементы справочников, упомянутые в файле: организации, контрагентов, склады (справочник «Структурные единицы») и номенклатуру всех строк. Поиск строгий (листинг 3): ищется точное совпадение наименования среди непомеченных на удаление элементов (для справочников с группами — только среди элементов, не групп). Запрос выбирает не больше двух записей: ни одной — ошибка «Не найден элемент

справочника…», две — ошибка «Несколько совпадений…». Найденные ссылки кэшируются в соответствии, чтобы не искать одно наименование повторно. Если есть хотя бы одна ошибка, документы не создаются вообще, а ошибки выводятся в протокол — так ошибка во второй накладной не оставит в базе уже созданную первую.

Листинг 3 — Строгий поиск элемента справочника по наименованию

```
// Строгий поиск элемента справочника по наименованию: ошибка, если элемент
// не найден или найдено несколько элементов с таким наименованием.
&НаСервереБезКонтекста
Процедура НайтиПоНаименованию(ИмяСправочника, Наименование, Найденные,
Ошибки)

    Ключ = ИмяСправочника + "|" + Наименование;
    Если Найденные[Ключ] <> Неопределено Тогда
        Возврат;
    КонецЕсли;

    МетаданныеСправочника = Метаданные.Справочники[ИмяСправочника];
    Если ПустаяСтрока(Наименование) Тогда
        ТекстОшибки = "Не заполнено наименование (справочник """ +
МетаданныеСправочника.Синоним + """).";
        Если Ошибки.Найти(ТекстОшибки) = Неопределено Тогда
            Ошибки.Добавить(ТекстОшибки);
        КонецЕсли;
        Возврат;
    КонецЕсли;

    ТекстЗапроса =
    "ВЫБРАТЬ ПЕРВЫЕ 2
    |    Т.Ссылка КАК Ссылка
    |ИЗ
    |    Справочник." + ИмяСправочника + " КАК Т
    |ГДЕ
    |    Т.Наименование = &Наименование
    |    И НЕ Т.ПометкаУдаления";
    Если МетаданныеСправочника.Иерархический
        И МетаданныеСправочника.ВидИерархии =
Метаданные.СвойстваОбъектов.ВидИерархии.ИерархияГруппИЭлементов Тогда
        ТекстЗапроса = ТекстЗапроса + "
        |    И НЕ Т.ЭтоГруппа";
    КонецЕсли;

    Запрос = Новый Запрос(ТекстЗапроса);
    Запрос.УстановитьПараметр("Наименование", Наименование);
    Выборка = Запрос.Выполнить().Выбрать();

    Если Не Выборка.Следующий() Тогда
        Ошибки.Добавить("Не найден элемент справочника """ +
МетаданныеСправочника.Синоним + """: " + Наименование);
        Возврат;
    КонецЕсли;
    Ссылка = Выборка.Ссылка;
    Если Выборка.Следующий() Тогда
        Ошибки.Добавить("Несколько совпадений в справочнике """ +
МетаданныеСправочника.Синоним + """: " + Наименование);
        Возврат;
```

```
    КонецЕсли;
    Найденные.Вставить(Ключ, Ссылка);
КонецПроцедуры
```

Шаг 4. Создание документов (листинг 4). Все документы создаются в одной транзакции, функцией НоваяРасходнаяНакладная:

– документ создается методом Документы.РасходнаяНакладная.СоздатьДокумент(), дата берется из файла (XMLЗначение);

– реквизиты шапки заполняются стандартным обработчиком заполнения УНФ (метод Заполнить) по структуре с видом операции «Продажа покупателю», организацией, контрагентом и структурной единицей. Этот обработчик сам заполняет обязательные поля, которых нет в файле: валюту, курс, налогообложение, вид цен, ответственного;

– договор подбирается функциями справочника «Договоры контрагентов»: по контрагенту, организации и виду операции или договор по умолчанию;

– для каждой строки «Товары» добавляется строка табличной части «Запасы»: номенклатура, единица измерения из карточки номенклатуры, количество и сумма из файла, цена = сумма / количество, ставка НДС по номенклатуре или организации, сумма НДС и итог строки;

– номер не заполняется — его присваивает платформа при записи (автонумерация УНФ с префиксами организации и информационной базы);

– документ записывается методом Записать(РежимЗаписиДокумента.Запись), то есть без проведения.

Если при записи любого документа возникает ошибка, транзакция отменяется и не остается ни одного документа. После успешного создания реквизит ТекстJSON очищается: повторное нажатие «Создать накладные» не создаст копии, для нового создания файл нужно загрузить заново. Методические указания отдельно предупреждают, что без такой защиты повторный импорт создает дубли.

Листинг 4 — Заполнение новой расходной накладной по объекту JSON

```
// Заполняет новую расходную накладную по объекту JSON. Номер не заполняется:
// его присвоит платформа (автонумерация) при записи.
&НаСервереБезКонтекста
Функция НоваяРасходнаяНакладная(ИсходныйДок, Найденные)

    Док = Документы.РасходнаяНакладная.СоздатьДокумент();
    Док.Дата = XMLЗначение(Тип("Дата"), ИсходныйДок.ДатаДокумента);

    ДанныеЗаполнения = Новый Структура;
    ДанныеЗаполнения.Вставить("ВидОперации",
Перечисления.ВидыОпераций РасходнаяНакладная.ПродажаПокупателю);
    ДанныеЗаполнения.Вставить("Организация", Найденные["Организации|" +
ИсходныйДок.Организация]);
    ДанныеЗаполнения.Вставить("Контрагент", Найденные["Контрагенты|" +
ИсходныйДок.Контрагент]);
    ДанныеЗаполнения.Вставить("СтруктурнаяЕдиница",
Найденные["СтруктурныеЕдиницы|" + ИсходныйДок.Склад]);

    // Заполнение по умолчанию средствами УНФ (валюта, курс, налогообложение,
ответственный и т.д.)
    Док.Заполнить(ДанныеЗаполнения);
    ЗаполнитьЗначенияСвойств(Док, ДанныеЗаполнения);
    Док.Дата = XMLЗначение(Тип("Дата"), ИсходныйДок.ДатаДокумента);

    Если Не ЗначениеЗаполнено(Док.Договор) Тогда
        Док.Договор =
Справочники.ДоговорыКонтрагентов.ПоДокументуКонтрагентуОрганизацииИВидуОперац
ии(
            Док, Док.Контрагент, Док.Организация, Док.ВидОперации);
    КонецЕсли;
    Если Не ЗначениеЗаполнено(Док.Договор) Тогда
        Док.Договор =
Справочники.ДоговорыКонтрагентов.ДоговорПоУмолчанию(Док.Контрагент);
    КонецЕсли;

    ВидСтавкиОрганизации =
ОбщегоНазначения.ЗначениеРеквизитаОбъекта(Док.Организация,
"ВидСтавкиНДСПоУмолчанию");
    Для Каждого Позиция Из ИсходныйДок.Товары Цикл

        Номенклатура = Найденные["Номенклатура|" + Позиция.Номенклатура];

        СтрокаТЧ = Док.Запасы.Добавить();
        СтрокаТЧ.Номенклатура = Номенклатура;
        СтрокаТЧ.ЕдиницаИзмерения = Номенклатура.ЕдиницаИзмерения;
        СтрокаТЧ.Количество = Позиция.Количество;
        СтрокаТЧ.Сумма = Позиция.Сумма;
        СтрокаТЧ.Цена = ?(СтрокаТЧ.Количество = 0, 0, Окр(СтрокаТЧ.Сумма /
СтрокаТЧ.Количество, 2));

        Если Док.НалогообложениеНДС =
Перечисления.ТипыНалогообложенияНДС.ОблагаетсяНДС Тогда
            ВидСтавки = ?(ЗначениеЗаполнено(Номенклатура.ВидСтавкиНДС),
Номенклатура.ВидСтавкиНДС, ВидСтавкиОрганизации);
            СтрокаТЧ.СтавкаНДС = Справочники.СтавкиНДС.СтавкаНДС(ВидСтавки,
Док.Дата);
            Ставка =
УправлениеНебольшойФирмойПовтИсп.ПолучитьЗначениеСтавкиНДС(СтрокаТЧ.СтавкаНДС
);
        Иначе
            СтрокаТЧ.СтавкаНДС =
УправлениеНебольшойФирмойПовтИсп.ПолучитьСтавкуНДСБезНДС();
```

```
            Ставка = 0;
        КонецЕсли;

        СтрокаТЧ.СуммаНДС = ?(Док.СуммаВключаетНДС,
            СтрокаТЧ.Сумма - СтрокаТЧ.Сумма / ((Ставка + 100) / 100),
            СтрокаТЧ.Сумма * Ставка / 100);
        СтрокаТЧ.Всего = СтрокаТЧ.Сумма + ?(Док.СуммаВключаетНДС, 0,
СтрокаТЧ.СуммаНДС);

    КонецЦикла;

    Док.СуммаДокумента = Док.Запасы.Итог("Всего");

    Возврат Док;

КонецФункции
```

Шаг 5. Проверка. Обработка запущена с файлом из задачи 1 в обеих базах. В базе «Филиал» созданы расходные накладные АСФЛ-000001 от 24.04.2024 (1 строка, 500,00 руб.) и АСФЛ-000002 от 26.04.2024 (21 строка, 879 828,00 руб.) (рисунок 6), в базе «Головной офис» — АСФР-000004 и АСФР-000005. Номера новые и не совпадают с номерами исходных приходных накладных: «АС» — префикс организации «Ассоль ООО», «ФЛ» и «ФР» — префиксы баз (подраздел 2.4).

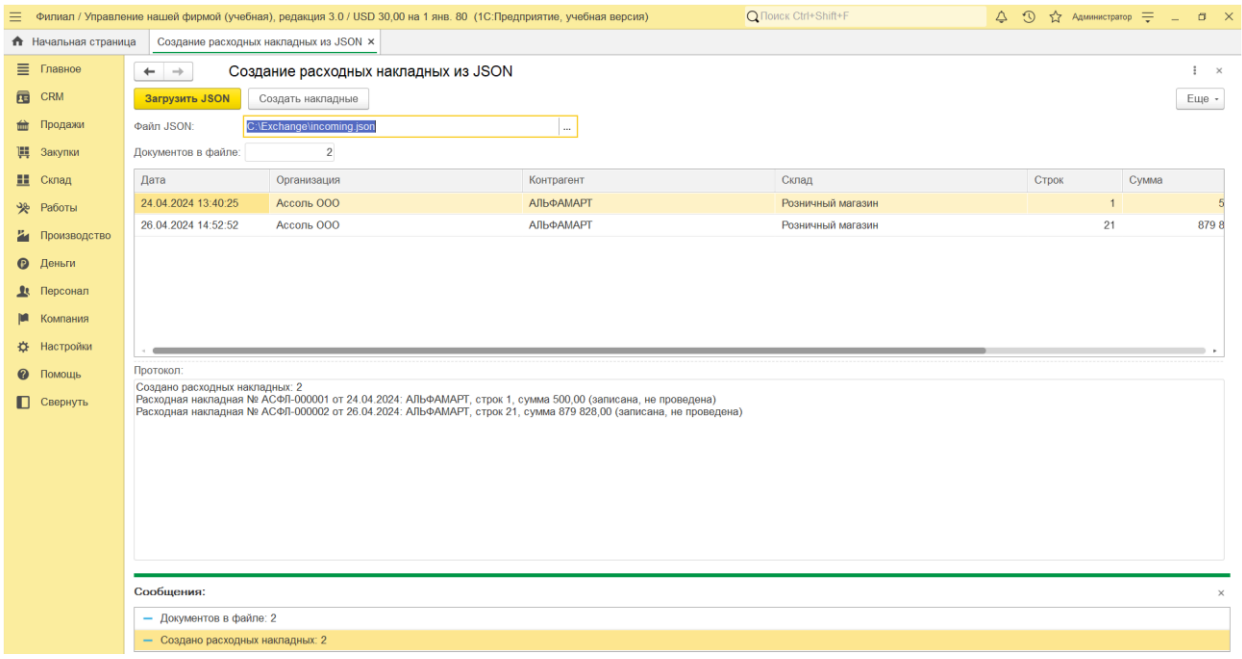


Рисунок 6 — Протокол создания расходных накладных в базе «Филиал»

В списке расходных накладных Головного офиса (рисунок 7) новые документы отмечены значком непроведенного документа — в отличие от проведенных документов демо-базы со значком проводки. В списке видны и

накладные АСФЛ-000001 и АСФЛ-000002, пришедшие из Филиала обменом (задача 3). Открытый документ АСФР-000005 (рисунок 8) имеет заголовок «Расходная накладная 5 от 26.04.2024 (не проведен)»; дата, организация, контрагент, склад и строки товаров совпадают с файлом.

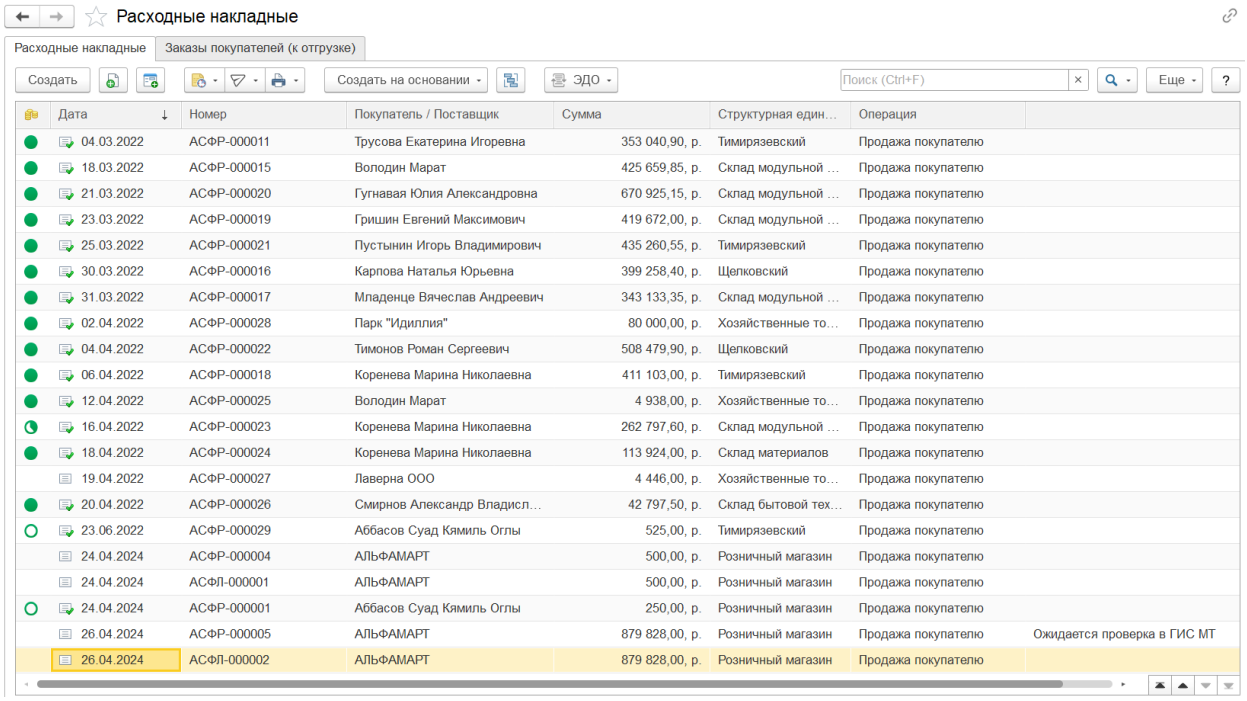


Рисунок 7 — Список расходных накладных в базе «Головной офис»

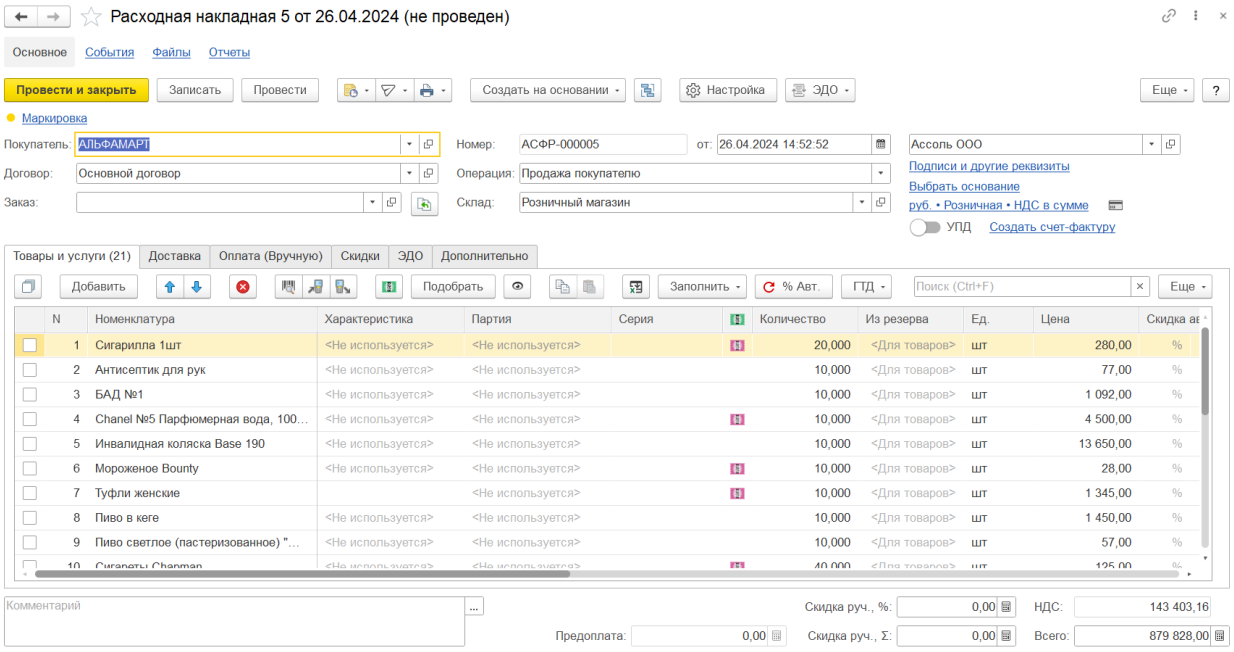


Рисунок 8 — Расходная накладная, созданная из JSON (не проведена)

Шаг 6. Проверка ошибки сопоставления. Как рекомендуют методические указания, проверено поведение при несуществующей

номенклатуре: в копии файла в первой строке второй накладной указана номенклатура «Несуществующий товар». Обработка вывела «Документы не созданы. Ошибки сопоставления: Не найден элемент справочника "Номенклатура": Несуществующий товар» и не создала ни одного документа, в том числе первую, корректную накладную (рисунок 9).

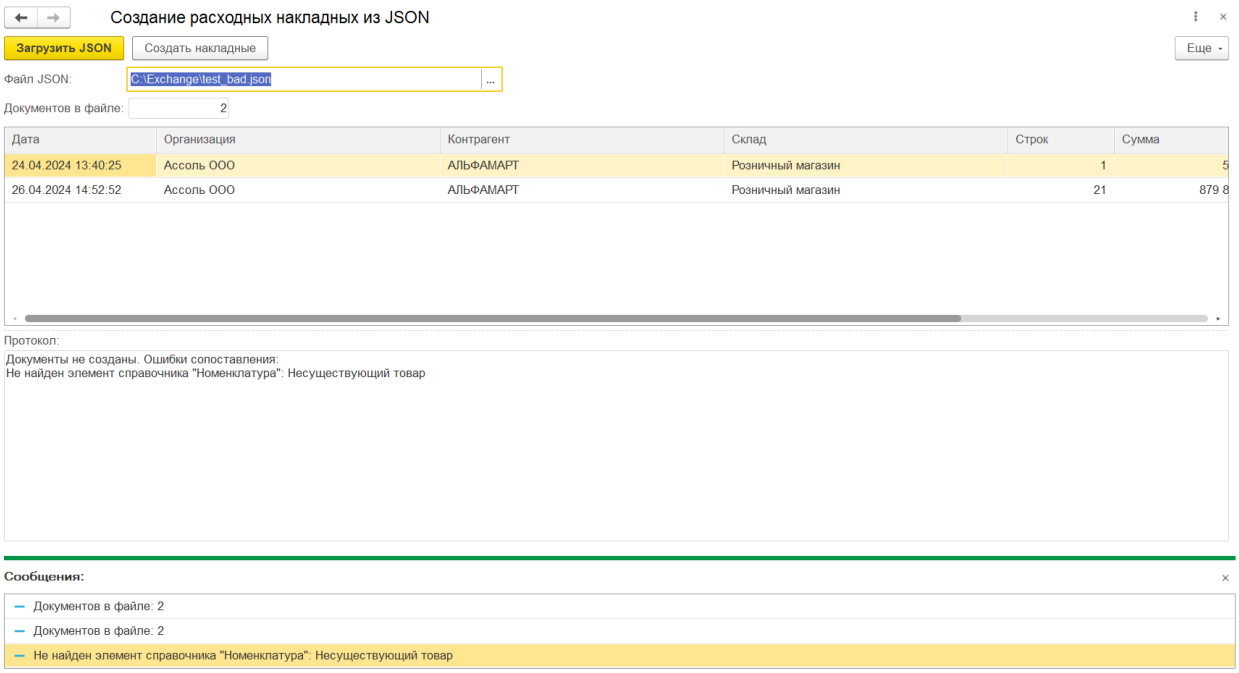


Рисунок 9 — Ошибка сопоставления: документы не созданы

2.4 Задача 3. Синхронизация баз «Головной офис» и «Филиал»

Методические указания предлагают два варианта: штатную синхронизацию данных (вариант А) и собственный файловый обмен (вариант Б). Задание разрешает любой способ; выбран вариант Б. Обмен выполняет внешняя обработка «ОбменФилиалГоловнойОфис». Она подключается к обеим базам как дополнительная обработка библиотеки стандартных подсистем (БСП) и запускается регламентным заданием по расписанию. Узлы обмена в конфигурации настраивать не нужно, а все правила — что передавать, как сопоставлять объекты и как избегать дублей — видны в коде обработки (приложение А).

Шаг 1. Вторая база. Базы должны быть разными, а не двумя ярлыками одной базы. Вторая база получена копированием каталога демонстрационной базы C:\Users\krist\1C\УНФ_Демо в каталог C:\Users\krist\1C\УНФ_Филиал

(копировать файловую базу нужно при закрытых сеансах). Новый каталог добавлен в список баз стартера 1С: «Добавить» → добавление в список существующей информационной базы. Роли баз приведены в таблице 5. Поскольку Филиал — копия Головного офиса, справочники в базах изначально совпадают вместе с уникальными идентификаторами ссылок; это упрощает сопоставление объектов (шаг 3 подготовки в методических указаниях).

Таблица 5 — Базы, участвующие в обмене

Роль	Каталог базы	Имя в списке баз	Заголовок	Префикс
Головной офис	C:\Users\krist\1C\УНФ_Демо	УНФ Головной офис (демо, практики 4-5)	Головной офис	ФР
Филиал	C:\Users\krist\1C\УНФ_Филиал	УНФ Филиал (практика 5)	Филиал	ФЛ

Шаг 2. Заголовок и префикс базы. Чтобы базы различались на экране и в номерах документов, в каждой базе заданы заголовок приложения («Настройки → Администрирование → Общие настройки», поле «Заголовок приложения», рисунок 10) и префикс информационной базы («Настройки → Интеграции → Синхронизация данных», поле «Префикс этой информационной базы», ссылка «Изменить», рисунок 11). Префикс входит в номера новых документов, поэтому документы из Филиала (АСФЛ-…) не конфликтуют по номерам с документами Головного офиса (АСФР-…). Заголовок приложения применяется после перезапуска клиента; на снимках Головного офиса в заголовке окна еще видна старая надпись «[КОПИЯ] Управление нашей фирмой».

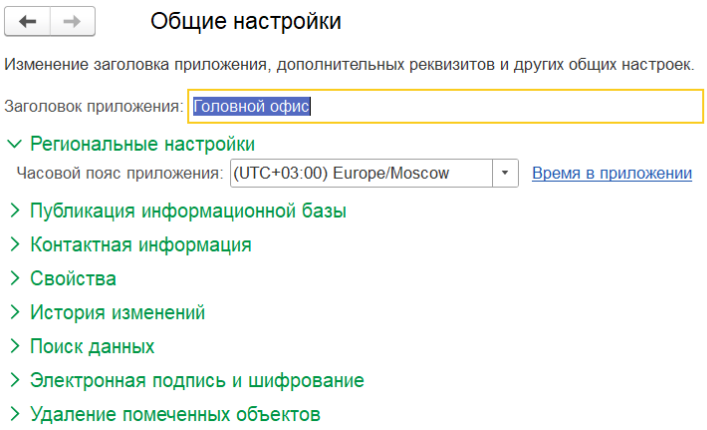


Рисунок 10 — Заголовок приложения в общих настройках

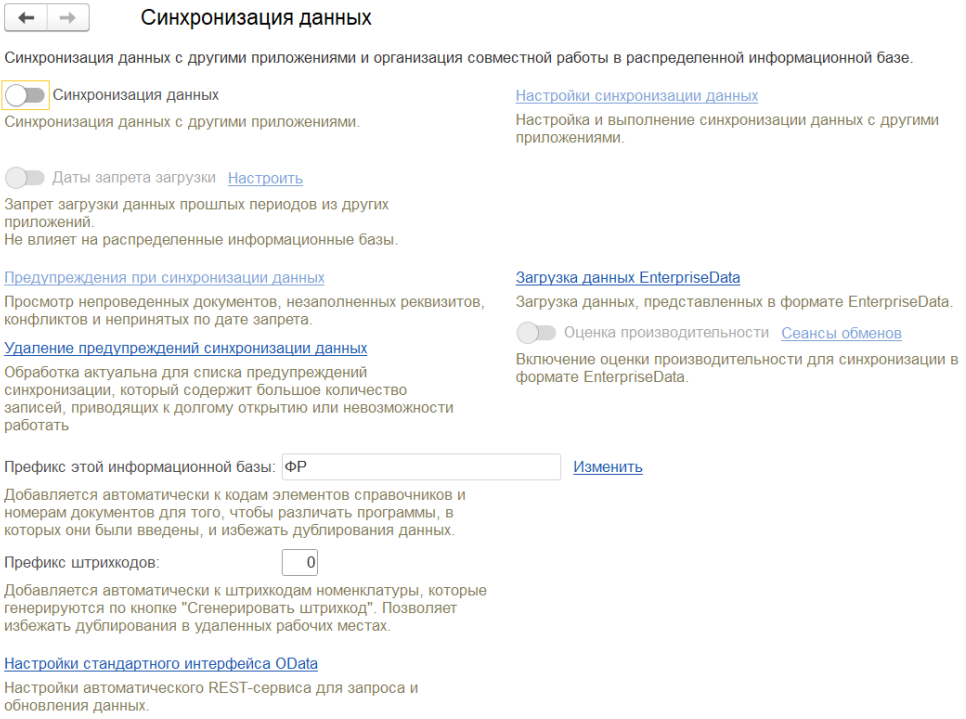


Рисунок 11 — Префикс информационной базы

Шаг 3. Разрешение работы с внешними ресурсами. Если база была перемещена или скопирована (при первом запуске демо-базы в практической работе №4 выбрано «Это копия информационной базы»), БСП блокирует в ней работу с внешними ресурсами. В такой базе регламентные задания отключены, и обмен по расписанию не запустится: задание отменяется без выполнения. Поэтому в обеих базах блокировка снята, а регламентное задание «Запуск дополнительных обработок» включено. В работе это сделано один раз небольшой служебной внешней обработкой с кодом из листинга 5.

Листинг 5 — Снятие блокировки работы с внешними ресурсами

```
// Выполняется в каждой базе один раз (служебная внешняя обработка, режим
"1С:Предприятие")
БлокировкаРаботыСВнешнимиРесурсами.РазрешитьРаботуСВнешнимиРесурсами();

// Регламентное задание "Запуск дополнительных обработок" после снятия
блокировки включается
Отбор = Новый Структура("Метаданные",
Метаданные.РегламентныеЗадания.ЗапускДополнительныхОбработок);
Для Каждого Задание Из РегламентныеЗадания.ПолучитьРегламентныеЗадания(Отбор)
Цикл
    Если Не Задание.Использование Тогда
        Задание.Использование = Истина;
        Задание.Записать();
    КонецЕсли;
КонецЦикла;
```

Шаг 4. Каталог обмена. Обе базы работают с общим каталогом C:\UNFExchange (таблица 6). Каталоги создаются обработкой автоматически при первом запуске (процедура ПодготовитьКаталоги). Для каждого направления — свой канал (таблица 7), как в схеме варианта Б методических указаний.

Таблица 6 — Структура каталога обмена C:\UNFExchange

Каталог или файл	Назначение
branch_to_head\incoming	сообщения Филиала, ожидающие загрузки в Головной офис
branch_to_head\processed, \error	успешно загруженные и ошибочные сообщения этого канала
head_to_branch\incoming, \processed, \error	то же для канала Головной офис → Филиал
state\<Роль>_<канал>_versions.json	снимок версий объектов на момент последней выгрузки
state\<Роль>_<канал>_received.json	идентификаторы уже загруженных сообщений (защита от дублей)
state\<Роль>_mapping.json	таблица соответствий внешних и внутренних ссылок
log\exchange.log	общий журнал обмена обеих баз

Таблица 7 — Каналы обмена

Канал	Отправитель → получатель	Содержимое
branch_to_head	Филиал → Головной офис	новые и измененные документы: приходные и расходные накладные, заказы покупателей и поставщикам, счета на оплату, поступления в кассу и расходы из кассы, перемещения запасов; связанные элементы справочников
head_to_branch	Головной офис → Филиал	новые и измененные элементы справочника «Номенклатура» и связанные элементы справочников

Шаг 5. Выгрузка. Процедура Выгрузить (листинг 6) определяет, какие объекты нужно отправить:

– запрос получает ссылку и ВерсияДанных всех объектов канала. ВерсияДанных меняется при каждой записи объекта, поэтому отличие от сохраненного снимка означает, что объект создан или изменен;

– при первом запуске снимка нет: текущее состояние только запоминается (начальная инициализация), и существующие данные демо-базы не передаются;

– найденные объекты помещаются в пакет — структуру с ключами Формат, Канал, Отправитель, ИдентификаторСообщения (новый УИД), ДатаСоздания, Объекты и Зависимости. Объекты передаются целиком и сериализуются в JSON методом СериализаторXDTO.ЗаписатьJSON: конфигурации баз одинаковые, поэтому получатель восстановит их без ручного описания полей;

– в Зависимости попадают элементы справочников, на которые ссылаются объекты (реквизиты, строки табличных частей, родитель, владелец), кроме служебных — пользователей, идентификаторов метаданных и присоединенных файлов;

– файл с именем <канал>_<дата_время>_<8 символов УИД>.json сначала пишется с расширением .tmp и только потом переименовывается: получатель читает только *.json и не увидит недописанный файл;

– снимок версий обновляется только после записи файла. Если выгрузка прервется, следующий цикл повторит ее.

Листинг 6 — Определение новых и измененных объектов при выгрузке

```
// Выгружает новые и измененные с прошлого раза объекты в файл канала.
// При первом запуске только запоминает текущее состояние (начальная
инициализация).
Процедура Выгрузить(Роль, Канал, ПолныеИмена, Протокол)

    ИмяСнимка = Роль + "_" + Канал + "_versions";
    Снимок = ПрочитатьСостояние(ИмяСнимка);

    Текущие = Новый Соответствие; // "ПолноеИмя|УИД" -> ВерсияДанных
    Ссылки = Новый Соответствие;  // "ПолноеИмя|УИД" -> Ссылка
    Для Каждого ПолноеИмя Из ПолныеИмена Цикл
        Запрос = Новый Запрос(
            "ВЫБРАТЬ
            |    Т.Ссылка КАК Ссылка,
            |    Т.ВерсияДанных КАК ВерсияДанных
            |ИЗ
            |    " + ПолноеИмя + " КАК Т");
        Выборка = Запрос.Выполнить().Выбрать();
        Пока Выборка.Следующий() Цикл
            Ключ = КлючСсылки(Выборка.Ссылка);
            Текущие.Вставить(Ключ, Выборка.ВерсияДанных);
            Ссылки.Вставить(Ключ, Выборка.Ссылка);
        КонецЦикла;
    КонецЦикла;

    Если Снимок = Неопределено Тогда
        ЗаписатьСостояние(ИмяСнимка, Текущие);
        ЗаписатьВЖурнал(Роль, Протокол, "Канал " + Канал + ": начальная
инициализация, зафиксировано объектов: "
            + Текущие.Количество() + ". Будут выгружаться объекты, созданные
или измененные после этого момента.");
        Возврат;
    КонецЕсли;

    Измененные = Новый Массив;
    Для Каждого КлючИЗначение Из Текущие Цикл
        Если Снимок[КлючИЗначение.Ключ] <> КлючИЗначение.Значение Тогда
            Измененные.Добавить(Ссылки[КлючИЗначение.Ключ]);
        КонецЕсли;
    КонецЦикла;

    Если Измененные.Количество() = 0 Тогда
        Протокол.Добавить(ВремяСобытия() + " Канал " + Канал + ": новых и
измененных объектов нет");
        Возврат;
    ...
```

Шаг 6. Загрузка. Процедура Загрузить обрабатывает файлы *.json из каталога incoming своего канала в порядке имен, то есть в порядке создания:

– идентификатор сообщения сверяется со списком загруженных (state\<Роль>_<канал>_received.json). Повторное сообщение не обрабатывается, файл переносится в processed, а в журнал пишется «уже было загружено ранее»;

– зависимости: если объект с тем же УИД уже есть или известен по таблице соответствий, используется он; иначе объект ищется по правилам — контрагенты по ИНН и КПП, остальные справочники по наименованию, только при единственном совпадении. Найденное соответствие запоминается в state\<Роль>_mapping.json, а если ничего не найдено, элемент создается с тем же УИД;

– основные объекты записываются с ОбменДанными.Загрузка = Истина (как принято при обмене: без проверок заполнения и прикладных обработчиков), ссылки в них заменяются на сопоставленные. Документ, проведенный в источнике, затем проводится обычным образом и формирует движения; непроведенный остается непроведенным;

– успешно загруженный файл переносится в processed, ошибочный — в error. Каждое событие пишется в общий журнал log\exchange.log и в журнал регистрации (событие «Обмен Филиал - Головной офис»).

Один цикл обмена (функция ВыполнитьОбмен) для Филиала — выгрузка документов, затем загрузка номенклатуры, для Головного офиса — наоборот. Выгрузка и загрузка выполняются в отдельных попытках, поэтому ошибка одной из них не мешает другой. Полный текст модуля приведен в приложении А.

Шаг 7. Подключение обработки и расписание. В каждой базе выполнено:

1. «Настройки → Администрирование → Печатные формы, отчеты и обработки»: включен флажок «Дополнительные отчеты и обработки» (рисунок 12).

2. По ссылке «Дополнительные отчеты и обработки» открыт список, нажата кнопка «Добавить из файла…» и выбран файл ОбменФилиалГоловнойОфис.epf; предупреждение безопасности подтверждено. БСП вызывает экспортную функцию СведенияОВнешнейОбработке модуля объекта (листинг 7) и заполняет карточку: наименование, вид «Дополнительная обработка», версию и три

команды. Надпись «Подключается не безопасно» означает, что обработка работает в небезопасном режиме (БезопасныйРежим = Ложь): ей нужен доступ к файлам каталога обмена.

3. Размещение — раздел «Настройки»: команды обработки доступны в «Настройки → Сервис → Дополнительные обработки».

4. В Головном офисе для команды «Обмен: Головной офис (отправить номенклатуру, получить документы)» установлен флажок в колонке с часами и задано расписание (рисунки 13, 14): вкладка «Дневное», «Повторять через» — 60 сек. Получилось расписание «каждый день; каждые 60 секунд». В Филиале то же сделано для команды «Обмен: Филиал (отправить документы, получить номенклатуру)» (рисунок 15).

5. Нажата кнопка «Записать и закрыть». По расписанию БСП создает регламентное задание «Запуск дополнительных обработок», которое вызывает экспортную процедуру ВыполнитьКоманду обработки с идентификатором команды.

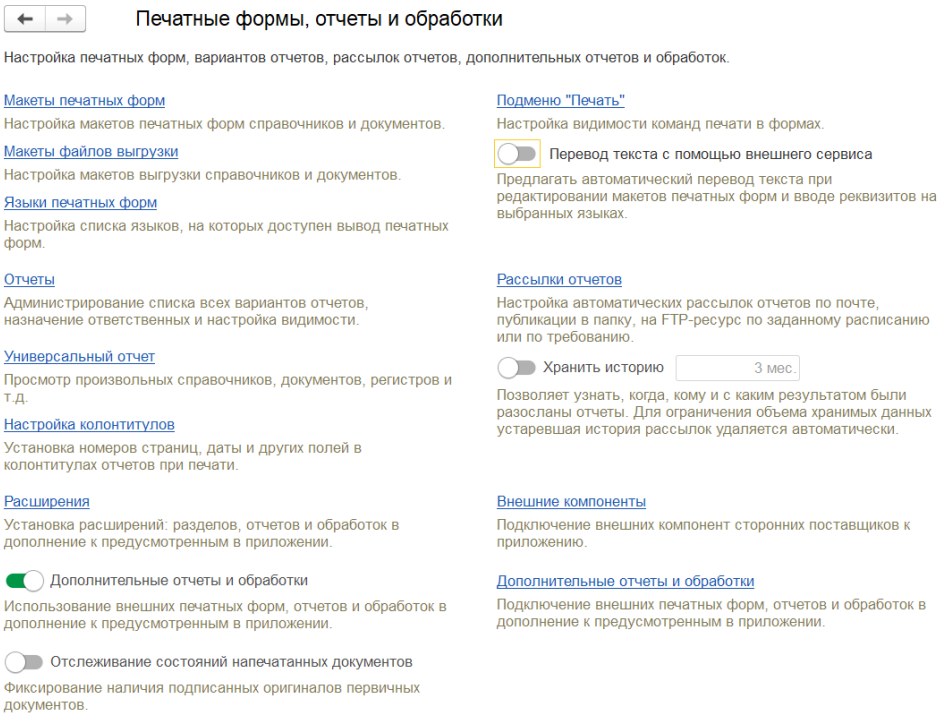


Рисунок 12 — Включение дополнительных отчетов и обработок

Листинг 7 — Регистрация обработки в БСП и выполнение команд

```
// Сведения для регистрации в справочнике "Дополнительные отчеты и обработки" (БСП).
Функция СведенияОВнешнейОбработке() Экспорт
```

```
    ПараметрыРегистрации =
ДополнительныеОтчетыИОбработки.СведенияОВнешнейОбработке("2.2.2.1");
    ПараметрыРегистрации.Вид =
ДополнительныеОтчетыИОбработкиКлиентСервер.ВидОбработкиДополнительнаяОбработк
а();
    ПараметрыРегистрации.Версия = "1.0";
    ПараметрыРегистрации.БезопасныйРежим = Ложь;
    ПараметрыРегистрации.Информация = "Обмен между базами УНФ ""Филиал"" и
""Головной офис"" через JSON-файлы в каталоге "
        + КаталогОбмена() + ". Филиал отправляет документы, Головной офис -
номенклатуру.";

    Команда = ПараметрыРегистрации.Команды.Добавить();
    Команда.Представление = "Обмен: Филиал (отправить документы, получить
номенклатуру)";
    Команда.Идентификатор = "ОбменФилиал";
    Команда.Использование =
ДополнительныеОтчетыИОбработкиКлиентСервер.ТипКомандыВызовСерверногоМетода();
    Команда.ПоказыватьОповещение = Истина;

    Команда = ПараметрыРегистрации.Команды.Добавить();
    Команда.Представление = "Обмен: Головной офис (отправить номенклатуру,
получить документы)";
    Команда.Идентификатор = "ОбменГоловнойОфис";
    Команда.Использование =
ДополнительныеОтчетыИОбработкиКлиентСервер.ТипКомандыВызовСерверногоМетода();
    Команда.ПоказыватьОповещение = Истина;

    Команда = ПараметрыРегистрации.Команды.Добавить();
    Команда.Представление = "Обмен Филиал - Головной офис (журнал и ручной
запуск)";
    Команда.Идентификатор = "ОткрытьФорму";
    Команда.Использование =
ДополнительныеОтчетыИОбработкиКлиентСервер.ТипКомандыОткрытиеФормы();

    Возврат ПараметрыРегистрации;

КонецФункции

// Вызывается БСП при выполнении серверной команды, в том числе регламентным
заданием по расписанию.
Процедура ВыполнитьКоманду(ИдентификаторКоманды, ПараметрыВыполнения =
Неопределено) Экспорт

    Если ИдентификаторКоманды = "ОбменФилиал" Тогда
        ВыполнитьОбмен("Филиал");
    ИначеЕсли ИдентификаторКоманды = "ОбменГоловнойОфис" Тогда
        ВыполнитьОбмен("ГоловнойОфис");
    КонецЕсли;

КонецПроцедуры
```

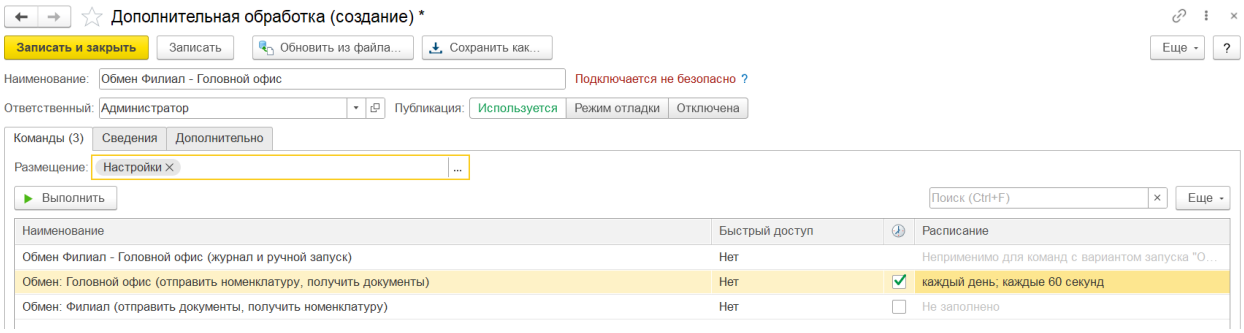


Рисунок 13 — Команды обработки обмена в базе «Головной офис»

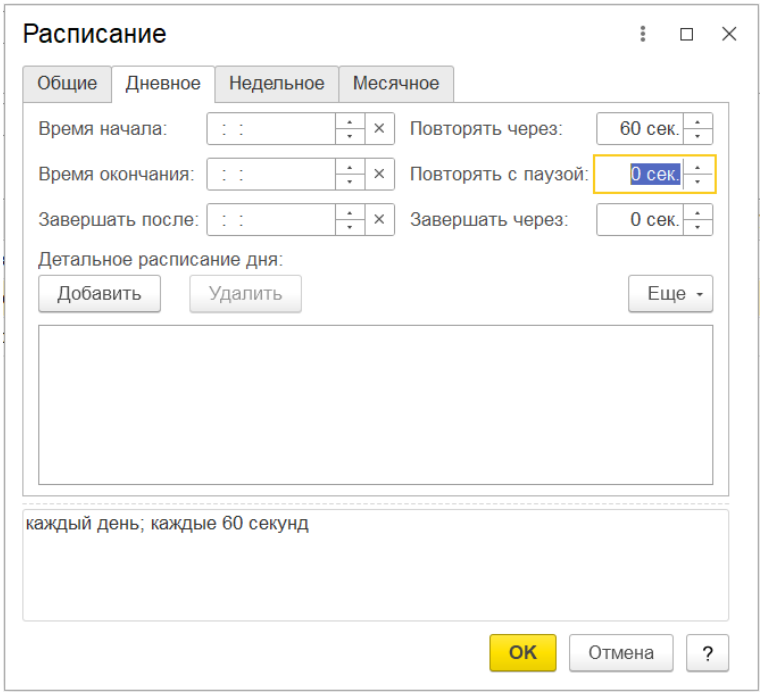


Рисунок 14 — Расписание запуска команды обмена

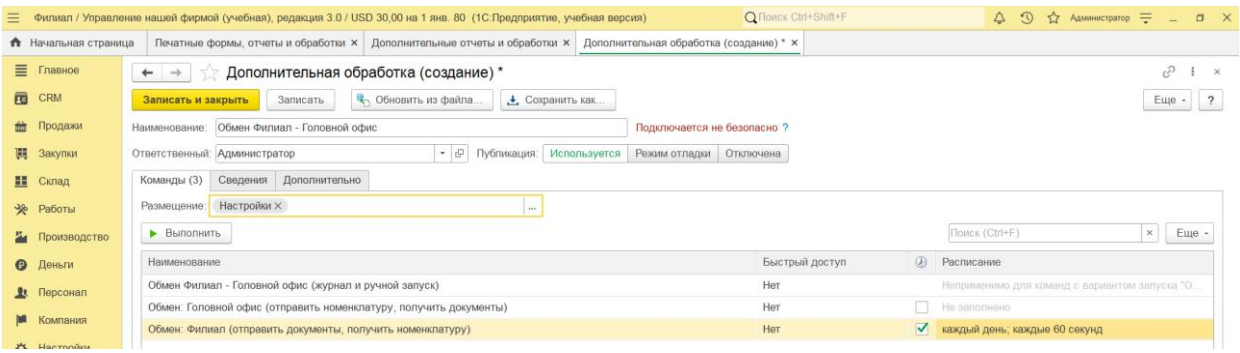


Рисунок 15 — Команды обработки обмена в базе «Филиал»

В файловом варианте регламентные задания выполняет клиентское приложение, поэтому обмен идет, пока открыт хотя бы один клиент каждой базы. Третья команда, «Обмен Филиал - Головной офис (журнал и ручной запуск)», открывает форму обработки (рисунок 18): на ней показан общий

журнал обмена, есть кнопка «Выполнить обмен сейчас» для ручного запуска и флажок автоматического обмена каждые 30 секунд, пока форма открыта.

Шаг 8. Проверка. Проведено контрольное испытание из методических указаний (таблица 8). Время событий взято из журнала обмена.

Таблица 8 — Результаты контрольного испытания обмена

Действие	Результат	Время
Первый запуск регламентных заданий	начальная инициализация: в Головном офисе зафиксировано 478 элементов номенклатуры, в Филиале — 2 214 документов	14:40:21, 15:02:48
В Головном офисе созданы «Чайник электрический 1,7 л (новинка ГО)» и «Кофеварка капельная (новинка ГО)»	каждая позиция выгружена в head_to_branch и примерно через минуту загружена в Филиал	15:05:39 → 15:06:36; 15:06:45 → 15:07:42
В Филиале созданы две расходные накладные (обработкой задачи 2)	выгружено 2 документа и 39 связанных элементов справочников; в Головном офисе созданы оба документа, не проведенные, как и в источнике	15:21:03 → 15:22:41
То же сообщение повторно помещено в каталог incoming	«сообщение … уже было загружено ранее, повторно не обрабатывается», дублей нет	15:24:08

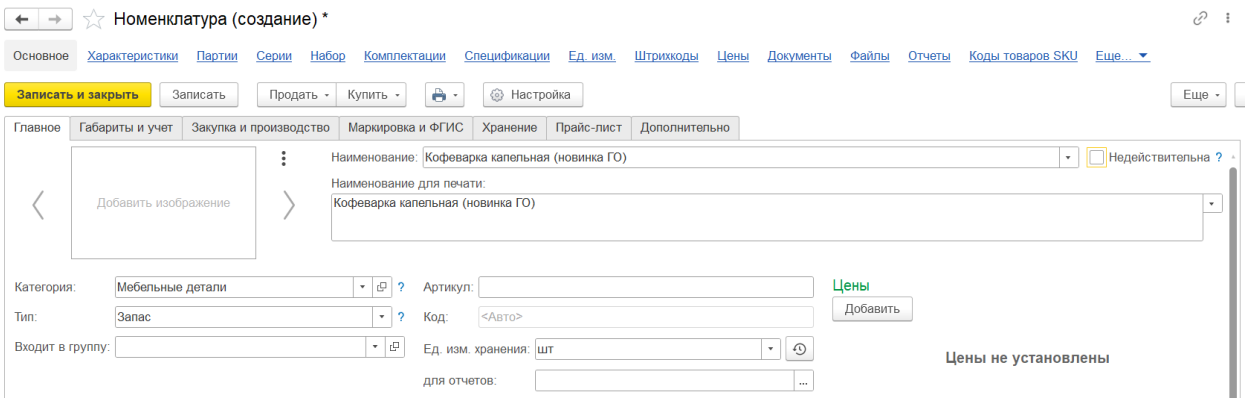


Рисунок 16 — Создание новой номенклатуры в базе «Головной офис»

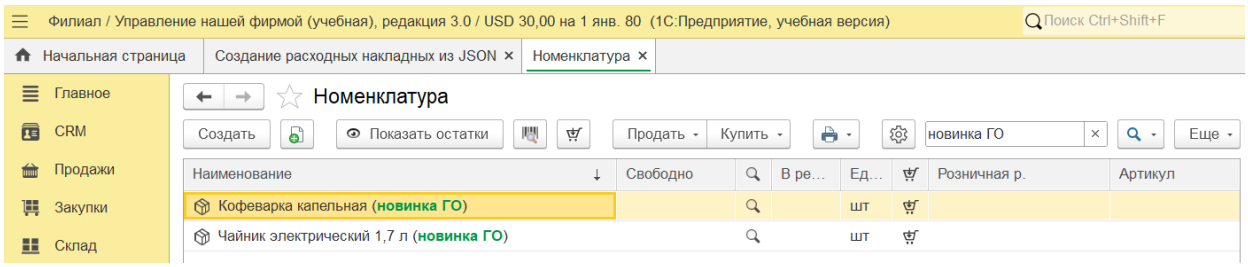


Рисунок 17 — Номенклатура из Головного офиса в базе «Филиал»

Новые позиции номенклатуры созданы в Головном офисе (рисунок 16) и появились в Филиале без участия пользователя (рисунок 17). Документы Филиала АСФЛ-000001 и АСФЛ-000002 появились в списке расходных накладных Головного офиса (рисунок 7). Все шаги отражены в журнале обмена (рисунок 18).

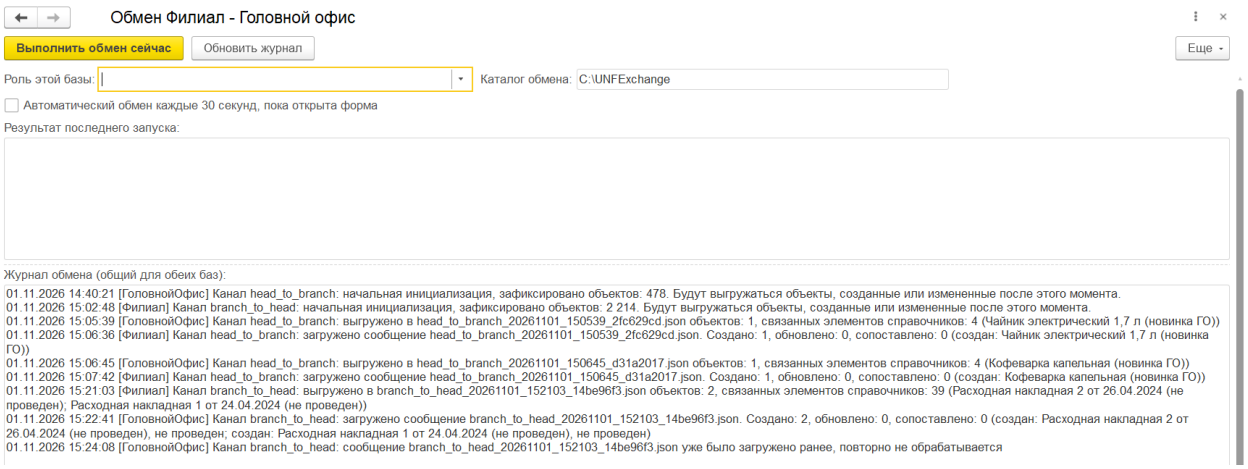


Рисунок 18 — Форма обработки обмена с журналом

Проверка с отключением доступа к каталогу не проводилась. На этот случай выгрузка и загрузка выполняются в отдельных попытках, ошибка пишется в журнал регистрации, входящие файлы не удаляются, а снимок версий не обновляется. Поэтому следующий цикл через 60 секунд повторит передачу.

2.5 Задача 4. Загрузка контрагентов из Excel

Шаг 1. Исходный файл. В файле «Пример файла для задачи 3.xlsx» один лист «Лист1», диапазон A1:E4: первая строка — заголовки, строки 2–4 — три контрагента (таблица 9).

Таблица 9 — Содержимое файла с контрагентами

A. Краткое наименование	B. Полное наименование	C. Вид	D. ИНН	E. КПП
ООО "Новый"	ОБЩЕСТВО С ОГРАНИЧЕННОЙ ОТВЕТСТВЕННОСТЬЮ "ГЕРМЕС"	Юридическое лицо	5406258962	520601001
ООО "Идеальный"	ОБЩЕСТВО С ОГРАНИЧЕННОЙ ОТВЕТСТВЕННОСТЬЮ "СТАЛЬОН"	Юридическое лицо	5905030487	520501001
ИП "Реквизит"	Индивидуальный предприниматель "Зырянов"	Индивидуальный предприниматель	5348740714	560501001

Колонки сопоставлены с реквизитами справочника «Контрагенты» так, как показано в таблице 10.

Таблица 10 — Сопоставление колонок файла и реквизитов справочника

Колонка	Реквизит	Правило
A	Наименование	«Наименование в программе»
B	НаименованиеПолное	«Наименование для документов»
C	ВидКонтрагента, ЮридическоеФизическоеЛицо	текст сравнивается с синонимами значений перечисления «Виды контрагентов» (юридическое лицо, индивидуальный предприниматель, физическое лицо, государственный орган)
D	ИНН	ключ поиска дублей; из ячейки берутся только цифры
E	КПП	заполняется только для юридических лиц и госорганов

Шаг 2. Форма. На форме обработки «ИмпортКонтрагентов» (синоним «Загрузка контрагентов из Excel») два реквизита: ПутьExcel (Строка, 500) с кнопкой выбора файла и РезультатИмпорта (Строка, многострочная, только просмотр). Команда «Загрузить» с заголовком «Загрузить контрагентов» размещена на командной панели. Если путь не заполнен, кнопка сначала открывает диалог выбора файла.

Шаг 3. Чтение и создание. Серверная функция ЗагрузитьExcelНаСервере:

– читает книгу встроенным объектом ТабличныйДокумент: Прочитать(ПутьExcel, СпособЧтенияЗначенийТабличногоДокумента.Значение), затем обходит

строки со второй до ВысотаТаблицы и берет текст ячеек методом Область(НомерСтроки, НомерКолонки).Текст;

– оставляет в ИНН и КПП только цифры (функция ТолькоЦифры): в Excel они хранятся числами, и при чтении могут появиться пробелы или разделители разрядов, а ИНН должен сравниваться как строка;

– пропускает пустые строки, а строки без ИНН пропускает с сообщением;

– ищет контрагента с таким ИНН запросом из листинга 8. Если он найден, элемент не создается: в протокол и в сообщения пользователю выводится «Строка N: контрагент с ИНН … уже существует (…), новый элемент не создан». Повтор ИНН внутри самого файла тоже считается дублем;

– определяет вид контрагента по синониму значения перечисления; если текст не распознан, ставит «Юридическое лицо» и пишет об этом в протокол;

– создает элемент: Справочники.Контрагенты.СоздатьЭлемент(), стандартное заполнение УНФ методом Заполнить (ответственный и другие значения по умолчанию), затем наименование, полное наименование, вид, юридическое или физическое лицо, ИНН и КПП, и записывает его методом Записать().

На клиенте каждое сообщение протокола выводится методом Сообщить, а если найдены дубли, дополнительно открывается окно предупреждения «Найдены дубли по ИНН» со списком (ПоказатьПредупреждение).

Листинг 8 — Поиск контрагента по ИНН

```
// Поиск дубликата по точному значению ИНН.
//
// Возвращаемое значение:
//  СправочникСсылка.Контрагенты - найденный контрагент или пустая ссылка.
//
&НаСервереБезКонтекста
Функция КонтрагентСИНН(ИНН)

    Запрос = Новый Запрос;
    Запрос.Текст =
```

```
    "ВЫБРАТЬ ПЕРВЫЕ 1
    |    Контрагенты.Ссылка КАК Ссылка
    |ИЗ
    |    Справочник.Контрагенты КАК Контрагенты
    |ГДЕ
    |    Контрагенты.ИНН = &ИНН
    |    И НЕ Контрагенты.ПометкаУдаления";
    Запрос.УстановитьПараметр("ИНН", ИНН);
    Выборка = Запрос.Выполнить().Выбрать();

    Если Выборка.Следующий() Тогда
        Возврат Выборка.Ссылка;
    КонецЕсли;

    Возврат Справочники.Контрагенты.ПустаяСсылка();

КонецФункции
```

Шаг 4. Проверка. При первой загрузке файла созданы три контрагента: ООО «Новый» и ООО «Идеальный» как юридические лица с КПП и ИП «Реквизит» как индивидуальный предприниматель. Для ИП КПП не применяется, поэтому значение 560501001 из файла не записано, о чем есть строка в протоколе. Карточка созданного контрагента показана на рисунке 19. При повторной загрузке того же файла новых элементов не появилось: обработка вывела «Создано: 0, дублей по ИНН: 3, пропущено: 0», сообщение по каждой строке и окно со списком дублей (рисунок 20). Обработка проверяет каждую строку отдельно, поэтому в файле, где есть и существующие, и новые контрагенты, создаются только новые, а существующие не перезаписываются.

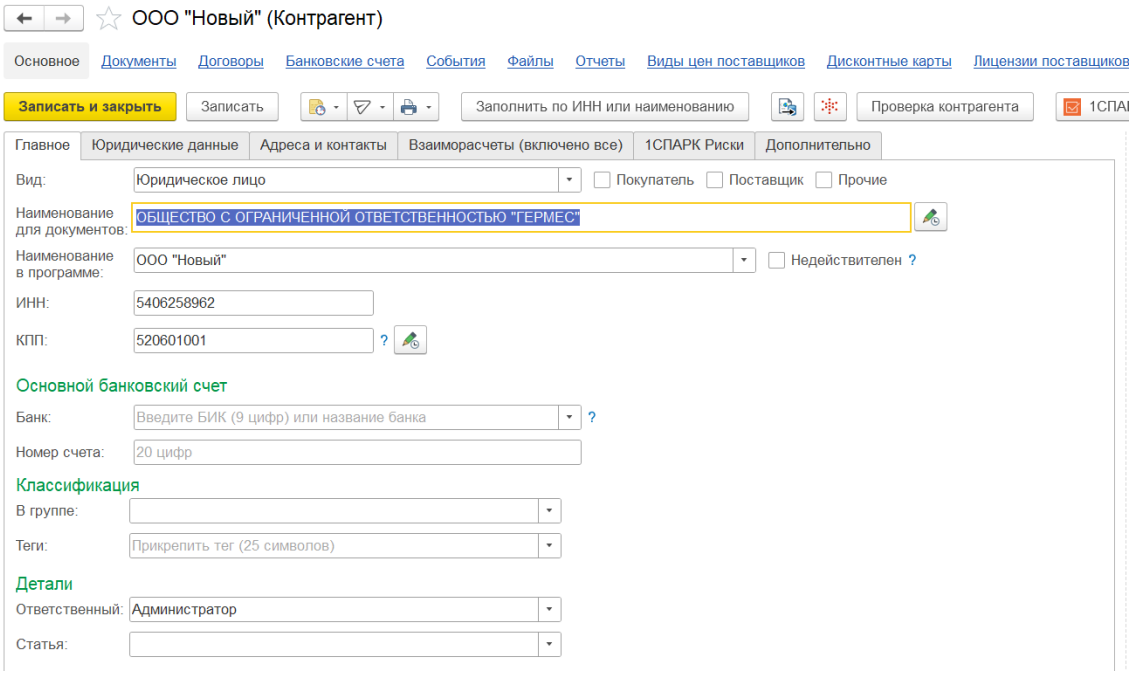


Рисунок 19 — Контрагент, созданный загрузкой из Excel

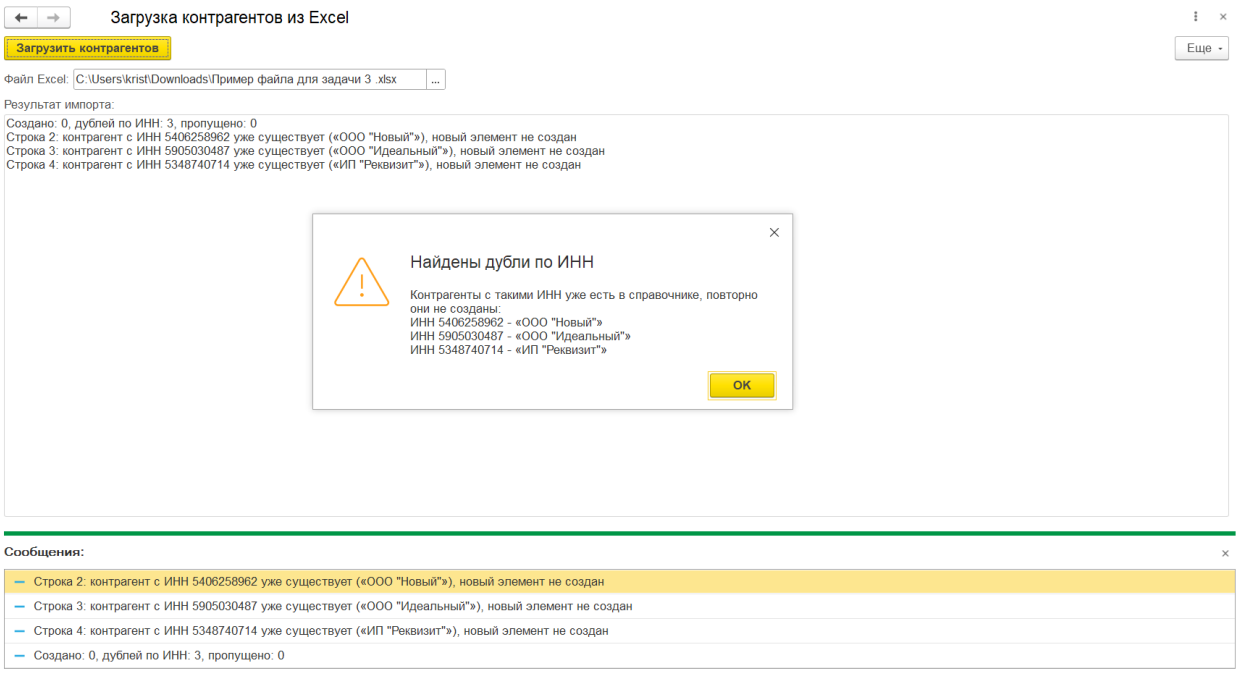


Рисунок 20 — Повторная загрузка: сообщения о дублях по ИНН

2.6 Задача 5*. HTTP-запрос GET и заглушка Mockoon

Шаг 1. Заглушка в Mockoon. Mockoon — бесплатная программа для создания локальных mock-серверов [4]. Использована переносимая версия 9.9.0 для Windows (файл mockoon.portable.9.9.0.exe с сайта mockoon.com или со страницы релизов проекта на GitHub), установка не требуется. В Mockoon создано окружение, как на снимке из задания:

1. Кнопкой «+» над списком окружений создать новое локальное окружение (New local environment) с именем Test и сохранить файл окружения (в работе — C:\Users\krist\Mockoon\Test.json).

2. На вкладке «Settings» окружения указать порт 3000 и префикс API «Test» — тогда все маршруты доступны по адресу http://localhost:3000/Test/…

3. На вкладке «Headers» окружения добавить заголовок Content-Type: application/json; charset=utf-8.

4. На вкладке «Routes» добавить маршрут GET с пустым путем «/», статусом 200 и телом {"testmessage": "Hello world!"}; маршрут GET «/TestPath» с JSON-ответом (сообщение и массив товаров); а также маршруты POST и DELETE для обоих путей, как на снимке из задания.

5. Запустить сервер зеленой кнопкой ▶ слева от вкладок (после запуска она становится красной ■).

Готовое окружение показано на рисунке 21. При первом запуске сервера Windows может спросить, разрешить ли Mockoon доступ к сетям. Для запросов с этого же компьютера (localhost) разрешение не нужно, поэтому достаточно нажать «Отменить».

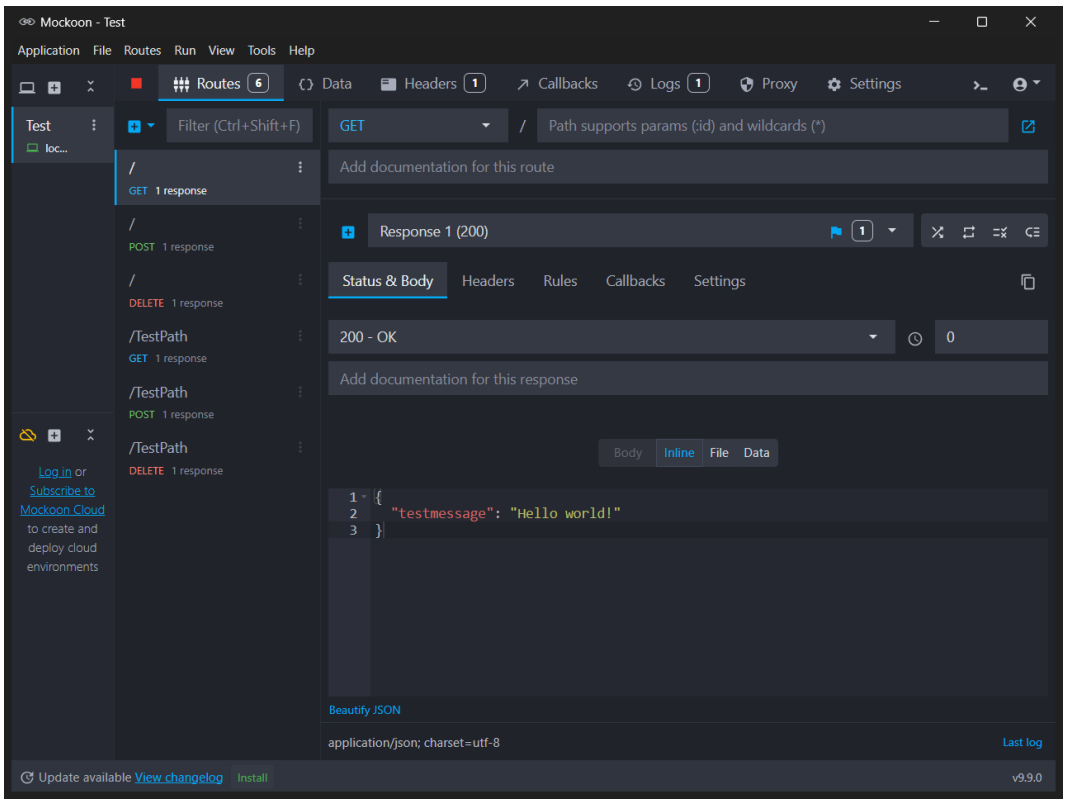


Рисунок 21 — Окружение Test в Mockoon: маршруты и ответ GET /

Шаг 2. Форма обработки. На форме обработки «HTTPЗапросGET» (синоним «HTTP-запрос GET») реквизиты из таблицы 11. Команда «ВыполнитьGET» с заголовком «GET» размещена кнопкой рядом с полем адреса и назначена кнопкой по умолчанию (срабатывает и по Ctrl+Enter). При открытии формы адрес заполняется значением http://localhost:3000/Test/.

Таблица 11 — Реквизиты формы обработки «HTTPЗапросGET»

Реквизит	Тип	Назначение
АдресURL	Строка (1000)	полный URL-адрес ресурса (заголовок поля «URL-адрес»)
КодСостояния	Число (3)	HTTP-код ответа, только просмотр
ВремяОтвета	Число (10)	время выполнения запроса в миллисекундах
Заголовки	Строка (неогранич.)	заголовки ответа, многострочное поле
ТелоОтвета	Строка (неогранич.)	тело ответа; JSON выводится с отступами

Шаг 3. Выполнение запроса (листинг 9). Объекту HTTPСоединение нужны отдельно хост и порт, а объекту HTTPЗапрос — путь с параметрами, поэтому функция РазобратьURL делит адрес на части. Она определяет схему (поддерживаются http и https), хост и порт (по умолчанию 80 или 443), путь

вместе со строкой параметров и отбрасывает фрагмент «#…». Затем выполняется:

– создание соединения: Новый HTTPСоединение(Хост, Порт, , , , 30, ЗащищенноеСоединение) с тайм-аутом 30 секунд; для https передается объект ЗащищенноеСоединениеOpenSSL;

– запрос методом Соединение.Получить(Новый HTTPЗапрос(ПутьИПараметры)) и замер времени функцией ТекущаяУниверсальнаяДатаВМиллисекундах;

– вывод кода состояния и всех заголовков ответа;

– чтение тела ответа как строки (в UTF-8, если сервер не указал кодировку). Если тип содержимого — JSON, тело переформатируется с отступами: оно читается в структуру, сохраняющую порядок ключей, а если ключи не подходят для структуры — в соответствие;

– любая ошибка (сервер недоступен, неверный адрес) перехватывается, и ее текст выводится в поле «Тело ответа».

Листинг 9 — Выполнение GET-запроса

```
&НаСервере
Процедура ВыполнитьGETНаСервере()

    КодСостояния = 0;
    ВремяОтвета = 0;
    Заголовки = "";
    ТелоОтвета = "";

    Попытка

        Адрес = РазобратьURL(АдресURL);
        Если Адрес.Схема = "https" Тогда
            Защищенное = Новый ЗащищенноеСоединениеOpenSSL();
        Иначе
            Защищенное = Неопределено;
        КонецЕсли;

        Соединение = Новый HTTPСоединение(Адрес.Хост, Адрес.Порт, , , , 30,
Защищенное);
        Запрос = Новый HTTPЗапрос(Адрес.ПутьИПараметры);

        Начало = ТекущаяУниверсальнаяДатаВМиллисекундах();
        Ответ = Соединение.Получить(Запрос);
        ВремяОтвета = ТекущаяУниверсальнаяДатаВМиллисекундах() - Начало;

        КодСостояния = Ответ.КодСостояния;

        СтрокиЗаголовков = Новый Массив;
        ТипСодержимого = "";
```

```
        // Имя "Заголовок" в модуле формы занято свойством формы, поэтому
переменная называется иначе
        Для Каждого ЗаголовокОтвета Из Ответ.Заголовки Цикл
            СтрокиЗаголовков.Добавить(ЗаголовокОтвета.Ключ + ": " +
ЗаголовокОтвета.Значение);
            Если НРег(ЗаголовокОтвета.Ключ) = "content-type" Тогда
                ТипСодержимого = НРег(ЗаголовокОтвета.Значение);
            КонецЕсли;
        КонецЦикла;
        Заголовки = СтрСоединить(СтрокиЗаголовков, Символы.ПС);

        Если СтрНайти(ТипСодержимого, "charset") > 0 Тогда
            Тело = Ответ.ПолучитьТелоКакСтроку();
        Иначе
            Тело = Ответ.ПолучитьТелоКакСтроку("UTF-8");
        КонецЕсли;
        Если Тело = Неопределено Тогда
            Тело = "";
        КонецЕсли;

        Если СтрНайти(ТипСодержимого, "json") > 0 Тогда
            ТелоОтвета = ОтформатироватьJSON(Тело);
        Иначе
            ТелоОтвета = Тело;
        КонецЕсли;

    Исключение
        ТелоОтвета = "Ошибка выполнения запроса: " +
КраткоеПредставлениеОшибки(ИнформацияОбОшибке());
        Сообщить(ТелоОтвета);
    КонецПопытки;

КонецПроцедуры
```

Шаг 4. Проверка. Запрос GET http://localhost:3000/Test/ вернул код 200 и тело {"testmessage": "Hello world!"} с заголовком Content-Type: application/json; charset=utf-8 (рисунок 22). Запрос к маршруту http://localhost:3000/Test/TestPath вернул JSON с сообщением и массивом товаров (рисунок 23). Запрос к пути, которого нет в окружении (например, http://localhost:3000/ без префикса Test), возвращает код 404.

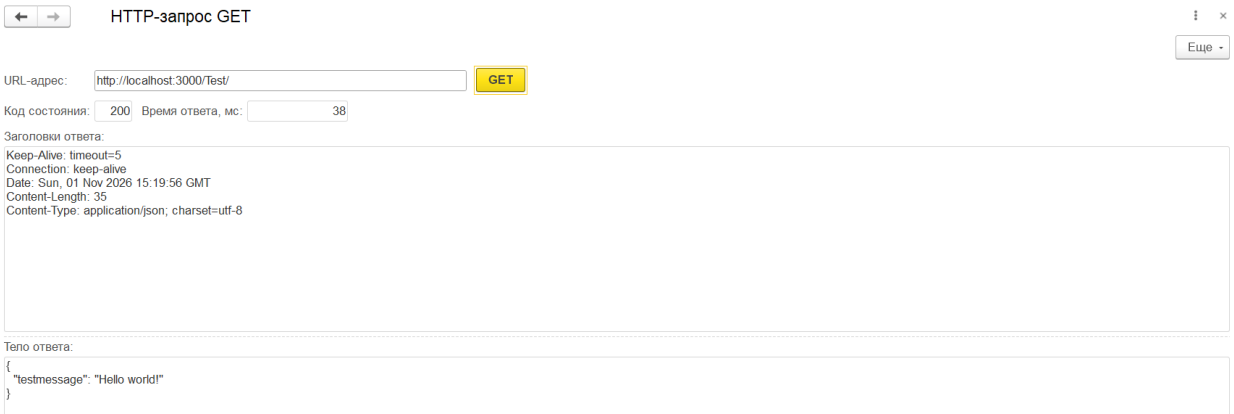


Рисунок 22 — Ответ заглушки на запрос GET /Test/

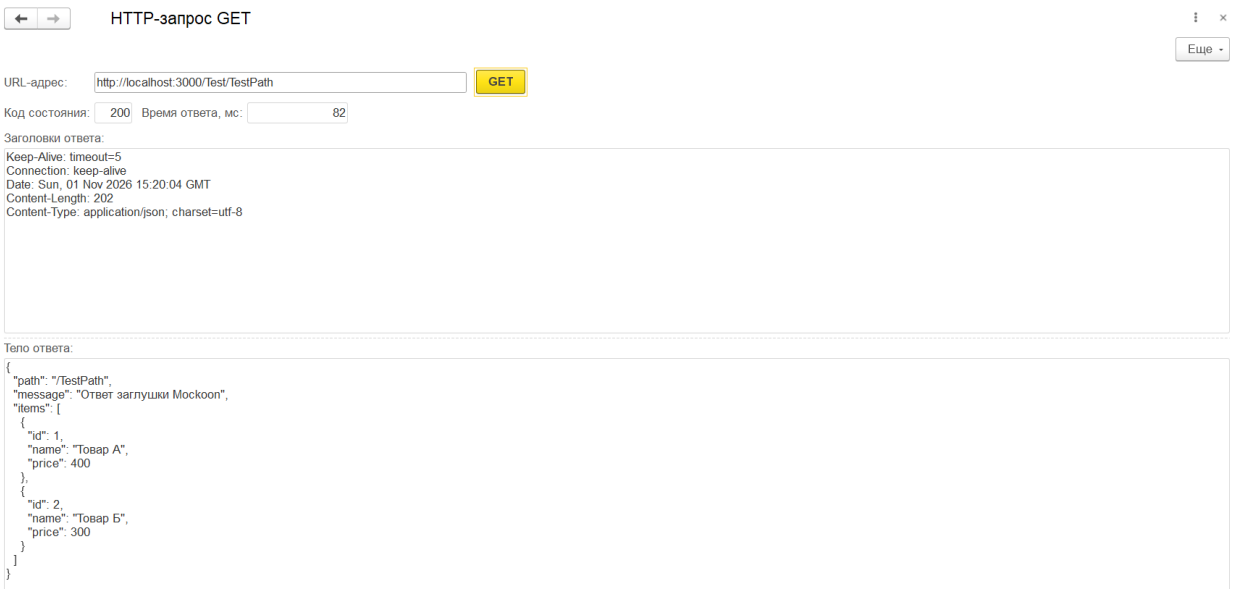


Рисунок 23 — Ответ заглушки на запрос GET /Test/TestPath

2.7 Особенности реализации

Ниже перечислено то, что сделано иначе, чем в методических указаниях, или дополнительно к заданию.

– Имена метаданных взяты из УНФ: табличная часть «Запасы», склад — «СтруктурнаяЕдиница», поставщик и покупатель — «Контрагент». Ключи JSON при этом универсальные, как требует задание;

– У расходной накладной есть обязательные реквизиты, которых нет в файле (вид операции, договор, валюта, налогообложение, единица измерения, ставка НДС). Они заполняются стандартным заполнением УНФ и по данным справочников, поэтому созданный документ можно сразу провести вручную;

– В задаче 2 все ссылки проверяются до создания первого документа, документы создаются в одной транзакции, а повторное нажатие «Создать накладные» без новой загрузки файла не создает копии;

– В задаче 3 выбран вариант Б (собственный файловый обмен). Изменения определяются по ВерсияДанных, объекты передаются целиком через сериализатор XDTO вместе со связанными элементами справочников, повторные сообщения отбрасываются по идентификатору, а документы проводятся в получателе, только если были проведены в источнике;

– Для обмена по расписанию в обеих базах пришлось снять блокировку работы с внешними ресурсами, которую БСП включает в копиях баз;

– В задаче 4 ИНН и КПП читаются как строки из цифр, КПП не заполняется для индивидуальных предпринимателей, а кроме дублей в базе выявляются и дубли внутри файла. О дублях пользователь узнает и из сообщений, и из отдельного окна предупреждения;

– В задаче 5 имена реквизита «URL» и переменной «Заголовок» в модуле формы использовать нельзя: это имена свойств самой формы (свойство «НавигационнаяСсылка» по-английски называется URL, а «Заголовок» — заголовок формы). С такими именами поле ввода не передавало адрес в код, а заголовок окна менялся на «КлючИЗначение». Поэтому реквизит назван «АдресURL», а переменная цикла — «ЗаголовокОтвета»;

– Ограничения учебной версии платформы: одно подключение к базе, поэтому для второй базы нужна отдельная копия. Список «Расходные накладные» в демо-базе открывается около 40 секунд.

ЗАКЛЮЧЕНИЕ

В ходе работы выполнены все задания, включая задание со звездочкой. В демонстрационной базе УНФ созданы пять внешних обработок:

– «ЭкспортНакладных» — выгрузка приходных накладных за период в JSON с вложенным массивом «Товары» (задача 1);

– «ИмпортРасходныхНакладных» — создание расходных накладных из этого файла со строгим сопоставлением справочников по наименованию, без проведения и с новыми номерами (задача 2);

– «ОбменФилиалГоловнойОфис» — автоматический обмен между базами «Головной офис» и «Филиал» через JSON-файлы по расписанию: документы идут из Филиала в Головной офис, номенклатура — обратно (задача 3);

– «ИмпортКонтрагентов» — загрузка контрагентов из Excel с контролем дублей по ИНН (задача 4);

– «HTTPЗапросGET» — выполнение GET-запроса по введенному адресу с выводом кода, заголовков и тела ответа; проверена на заглушке Mockoon (задача 5*).

Результаты проверены на данных демонстрационной базы. Выгрузка, создание документов и загрузка контрагентов дают ожидаемый результат, а ошибочные данные не приводят к частичной загрузке. Обмен передал новые позиции номенклатуры и документы без участия пользователя и не создал дублей при повторной обработке сообщения.

В процессе работы освоены чтение и запись JSON, в том числе сериализация объектов XDTO, чтение файлов Excel табличным документом, программное создание документов и элементов справочников с заполнением по умолчанию, подключение дополнительных обработок БСП с выполнением по расписанию и работа с HTTP-ресурсами из 1С. Эти приемы позволяют связывать базы 1С между собой и с внешними системами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Борисов, А. Ю. Практика 5. Интеграции : методические указания к практической работе по дисциплине «ВПК-8. Профессиональная разработка на платформе 1С:Предприятие» / А. Ю. Борисов. – Таганрог : ИКТИБ ЮФУ, 2026. – 59 с. – Текст : электронный.

2. 1С:Предприятие 8. Руководство разработчика. – Текст : электронный // ИТС : информационная система 1С. – URL: https://its.1c.ru (дата обращения: 01.11.2026).

3. Библиотека стандартных подсистем. Подсистема «Дополнительные отчеты и обработки». – Текст : электронный // ИТС : информационная система 1С. – URL: https://its.1c.ru (дата обращения: 01.11.2026).

4. Mockoon : документация. – Текст : электронный. – URL: https://mockoon.com/docs/latest/about/ (дата обращения: 01.11.2026).

5. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – 32 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Модуль объекта обработки «ОбменФилиалГоловнойОфис»

```
#Если Сервер Или ТолстыйКлиентОбычноеПриложение Или ВнешнееСоединение Тогда

// Обмен данными между двумя базами УНФ через JSON-файлы в общем каталоге.
//
// Канал branch_to_head: Филиал -> Головной офис, новые и измененные
документы.
// Канал head_to_branch: Головной офис -> Филиал, новые и измененные элементы
номенклатуры.
//
// Новые и измененные объекты определяются сравнением ВерсияДанных с
сохраненным снимком.
// Объекты сериализуются средствами XDTO (конфигурации баз одинаковые),
идентификатором обмена
// служит уникальный идентификатор ссылки. Вместе с объектами передаются
элементы справочников,
// на которые они ссылаются; в базе-получателе отсутствующие элементы
сопоставляются по правилам
// (Контрагенты - по ИНН и КПП, остальные - по наименованию) или создаются.

#Область ПрограммныйИнтерфейс

// Сведения для регистрации в справочнике "Дополнительные отчеты и обработки"
(БСП).
Функция СведенияОВнешнейОбработке() Экспорт

    ПараметрыРегистрации =
ДополнительныеОтчетыИОбработки.СведенияОВнешнейОбработке("2.2.2.1");
    ПараметрыРегистрации.Вид =
ДополнительныеОтчетыИОбработкиКлиентСервер.ВидОбработкиДополнительнаяОбработк
а();
    ПараметрыРегистрации.Версия = "1.0";
    ПараметрыРегистрации.БезопасныйРежим = Ложь;
    ПараметрыРегистрации.Информация = "Обмен между базами УНФ ""Филиал"" и
""Головной офис"" через JSON-файлы в каталоге "
        + КаталогОбмена() + ". Филиал отправляет документы, Головной офис -
номенклатуру.";

    Команда = ПараметрыРегистрации.Команды.Добавить();
    Команда.Представление = "Обмен: Филиал (отправить документы, получить
номенклатуру)";
    Команда.Идентификатор = "ОбменФилиал";
    Команда.Использование =
ДополнительныеОтчетыИОбработкиКлиентСервер.ТипКомандыВызовСерверногоМетода();
    Команда.ПоказыватьОповещение = Истина;

    Команда = ПараметрыРегистрации.Команды.Добавить();
    Команда.Представление = "Обмен: Головной офис (отправить номенклатуру,
получить документы)";
    Команда.Идентификатор = "ОбменГоловнойОфис";
    Команда.Использование =
ДополнительныеОтчетыИОбработкиКлиентСервер.ТипКомандыВызовСерверногоМетода();
    Команда.ПоказыватьОповещение = Истина;

    Команда = ПараметрыРегистрации.Команды.Добавить();
    Команда.Представление = "Обмен Филиал - Головной офис (журнал и ручной
запуск)";
```

```
    Команда.Идентификатор = "ОткрытьФорму";
    Команда.Использование =
ДополнительныеОтчетыИОбработкиКлиентСервер.ТипКомандыОткрытиеФормы();

    Возврат ПараметрыРегистрации;

КонецФункции

// Вызывается БСП при выполнении серверной команды, в том числе регламентным
заданием по расписанию.
Процедура ВыполнитьКоманду(ИдентификаторКоманды, ПараметрыВыполнения =
Неопределено) Экспорт

    Если ИдентификаторКоманды = "ОбменФилиал" Тогда
        ВыполнитьОбмен("Филиал");
    ИначеЕсли ИдентификаторКоманды = "ОбменГоловнойОфис" Тогда
        ВыполнитьОбмен("ГоловнойОфис");
    КонецЕсли;

КонецПроцедуры

// Выполняет один цикл обмена для базы с указанной ролью: сначала выгрузка,
затем загрузка.
//
// Параметры:
//  Роль - Строка - "Филиал" или "ГоловнойОфис".
//
// Возвращаемое значение:
//  Строка - протокол цикла обмена.
//
Функция ВыполнитьОбмен(Роль) Экспорт

    Протокол = Новый Массив;

    Если Роль = "Филиал" Тогда
        КаналВыгрузки = "branch_to_head";
        КаналЗагрузки = "head_to_branch";
        ОбъектыВыгрузки = ДокументыФилиала();
    ИначеЕсли Роль = "ГоловнойОфис" Тогда
        КаналВыгрузки = "head_to_branch";
        КаналЗагрузки = "branch_to_head";
        ОбъектыВыгрузки = СтрРазделить("Справочник.Номенклатура", ",");
    Иначе
        ВызватьИсключение "Неизвестная роль базы: " + Роль;
    КонецЕсли;

    Попытка
        ПодготовитьКаталоги();
        Выгрузить(Роль, КаналВыгрузки, ОбъектыВыгрузки, Протокол);
    Исключение
        ЗаписатьВЖурнал(Роль, Протокол, "Выгрузка (" + КаналВыгрузки + ") не
выполнена: "
            + КраткоеПредставлениеОшибки(ИнформацияОбОшибке()), Истина);
    КонецПопытки;

    Попытка
        Загрузить(Роль, КаналЗагрузки, Протокол);
    Исключение
        ЗаписатьВЖурнал(Роль, Протокол, "Загрузка (" + КаналЗагрузки + ") не
выполнена: "
            + КраткоеПредставлениеОшибки(ИнформацияОбОшибке()), Истина);
    КонецПопытки;

    Возврат СтрСоединить(Протокол, Символы.ПС);
```

```
КонецФункции

// Общий каталог обмена, доступный обеим базам.
Функция КаталогОбмена() Экспорт

    Возврат "C:\UNFExchange";

КонецФункции

// Роль этой базы, сохраненная при последнем ручном запуске (по строке
соединения).
Функция РольЭтойБазы() Экспорт

    Узлы = ПрочитатьСостояние("nodes");
    Если Узлы = Неопределено Тогда
        Возврат "";
    КонецЕсли;
    Роль = Узлы[СтрокаСоединенияИнформационнойБазы()];
    Возврат ?(Роль = Неопределено, "", Роль);

КонецФункции

Процедура СохранитьРольЭтойБазы(Роль) Экспорт

    ПодготовитьКаталоги();
    Узлы = ПрочитатьСостояние("nodes");
    Если Узлы = Неопределено Тогда
        Узлы = Новый Соответствие;
    КонецЕсли;
    Узлы.Вставить(СтрокаСоединенияИнформационнойБазы(), Роль);
    ЗаписатьСостояние("nodes", Узлы);

КонецПроцедуры

// Последние строки общего журнала обмена.
Функция ПрочитатьЖурнал(КоличествоСтрок = 60) Экспорт

    ИмяФайла = КаталогОбмена() + "\log\exchange.log";
    Файл = Новый Файл(ИмяФайла);
    Если Не Файл.Существует() Тогда
        Возврат "";
    КонецЕсли;

    Чтение = Новый ЧтениеТекста(ИмяФайла, КодировкаТекста.UTF8);
    Текст = Чтение.Прочитать();
    Чтение.Закрыть();

    Строки = СтрРазделить(Текст, Символы.ПС, Ложь);
    Последние = Новый Массив;
    Для Индекс = Макс(0, Строки.Количество() - КоличествоСтрок) По
Строки.ВГраница() Цикл
        Последние.Добавить(Строки[Индекс]);
    КонецЦикла;
    Возврат СтрСоединить(Последние, Символы.ПС);

КонецФункции

#КонецОбласти

#Область Выгрузка

Функция ДокументыФилиала()
```

```
    Результат = Новый Массив;
    Имена =
"ПриходнаяНакладная,РасходнаяНакладная,ЗаказПокупателя,ЗаказПоставщику,СчетНа
Оплату,"
        + "ПоступлениеВКассу,РасходИзКассы,ПеремещениеЗапасов";
    Для Каждого Имя Из СтрРазделить(Имена, ",") Цикл
        Если Метаданные.Документы.Найти(Имя) <> Неопределено Тогда
            Результат.Добавить("Документ." + Имя);
        КонецЕсли;
    КонецЦикла;
    Возврат Результат;

КонецФункции

// Выгружает новые и измененные с прошлого раза объекты в файл канала.
// При первом запуске только запоминает текущее состояние (начальная
инициализация).
Процедура Выгрузить(Роль, Канал, ПолныеИмена, Протокол)

    ИмяСнимка = Роль + "_" + Канал + "_versions";
    Снимок = ПрочитатьСостояние(ИмяСнимка);

    Текущие = Новый Соответствие; // "ПолноеИмя|УИД" -> ВерсияДанных
    Ссылки = Новый Соответствие;  // "ПолноеИмя|УИД" -> Ссылка
    Для Каждого ПолноеИмя Из ПолныеИмена Цикл
        Запрос = Новый Запрос(
            "ВЫБРАТЬ
            |    Т.Ссылка КАК Ссылка,
            |    Т.ВерсияДанных КАК ВерсияДанных
            |ИЗ
            |    " + ПолноеИмя + " КАК Т");
        Выборка = Запрос.Выполнить().Выбрать();
        Пока Выборка.Следующий() Цикл
            Ключ = КлючСсылки(Выборка.Ссылка);
            Текущие.Вставить(Ключ, Выборка.ВерсияДанных);
            Ссылки.Вставить(Ключ, Выборка.Ссылка);
        КонецЦикла;
    КонецЦикла;

    Если Снимок = Неопределено Тогда
        ЗаписатьСостояние(ИмяСнимка, Текущие);
        ЗаписатьВЖурнал(Роль, Протокол, "Канал " + Канал + ": начальная
инициализация, зафиксировано объектов: "
            + Текущие.Количество() + ". Будут выгружаться объекты, созданные
или измененные после этого момента.");
        Возврат;
    КонецЕсли;

    Измененные = Новый Массив;
    Для Каждого КлючИЗначение Из Текущие Цикл
        Если Снимок[КлючИЗначение.Ключ] <> КлючИЗначение.Значение Тогда
            Измененные.Добавить(Ссылки[КлючИЗначение.Ключ]);
        КонецЕсли;
    КонецЦикла;

    Если Измененные.Количество() = 0 Тогда
        Протокол.Добавить(ВремяСобытия() + " Канал " + Канал + ": новых и
измененных объектов нет");
        Возврат;
    КонецЕсли;

    Пакет = Новый Структура;
    Пакет.Вставить("Формат", "UNFExchange/1.0");
    Пакет.Вставить("Канал", Канал);
```

```
    Пакет.Вставить("Отправитель", Роль);
    Пакет.Вставить("ИдентификаторСообщения", Строка(Новый
УникальныйИдентификатор));
    Пакет.Вставить("ДатаСоздания", ТекущаяДатаСеанса());
    Пакет.Вставить("Объекты", Новый Массив);
    Пакет.Вставить("Зависимости", Новый Массив);

    Добавленные = Новый Соответствие;
    Представления = Новый Массив;
    Для Каждого Ссылка Из Измененные Цикл
        Объект = Ссылка.ПолучитьОбъект();
        Если Объект <> Неопределено Тогда
            Пакет.Объекты.Добавить(Объект);
            Добавленные.Вставить(Ссылка, Истина);
            Представления.Добавить(Строка(Ссылка));
        КонецЕсли;
    КонецЦикла;
    Для Каждого Объект Из Пакет.Объекты Цикл
        СобратьЗависимости(Объект, Пакет.Зависимости, Добавленные);
    КонецЦикла;

    ИмяФайла = Канал + "_" + Формат(ТекущаяДатаСеанса(),
"ДФ=yyyyMMdd_HHmmss") + "_"
        + Лев(Пакет.ИдентификаторСообщения, 8) + ".json";
    КаталогВходящих = КаталогОбмена() + "\" + Канал + "\incoming\";
    ВременныйФайл = КаталогВходящих + ИмяФайла + ".tmp";

    Запись = Новый ЗаписьJSON;
    Запись.ОткрытьФайл(ВременныйФайл, "UTF-8", Ложь, Новый
ПараметрыЗаписиJSON(ПереносСтрокJSON.Авто, Символы.Таб));
    СериализаторXDTO.ЗаписатьJSON(Запись, Пакет, НазначениеТипаXML.Явное);
    Запись.Закрыть();
    // Получатель читает только *.json, поэтому недописанный файл он не
увидит
    ПереместитьФайл(ВременныйФайл, КаталогВходящих + ИмяФайла);

    ЗаписатьСостояние(ИмяСнимка, Текущие);

    ЗаписатьВЖурнал(Роль, Протокол, "Канал " + Канал + ": выгружено в " +
ИмяФайла + " объектов: "
        + Пакет.Объекты.Количество() + ", связанных элементов справочников: "
+ Пакет.Зависимости.Количество()
        + " (" + СтрСоединить(Представления, "; ") + ")");

КонецПроцедуры

// Добавляет в пакет элементы справочников, на которые ссылается объект
(реквизиты, родитель,
// владелец, табличные части), чтобы получатель мог создать недостающие.
Процедура СобратьЗависимости(Объект, Зависимости, Добавленные)

    МетаданныеОбъекта = Объект.Метаданные();
    Значения = Новый Массив;

    Для Каждого Реквизит Из МетаданныеОбъекта.Реквизиты Цикл
        Значения.Добавить(Объект[Реквизит.Имя]);
    КонецЦикла;
    Если ОбщегоНазначения.ЭтоСправочник(МетаданныеОбъекта) Тогда
        Если МетаданныеОбъекта.Иерархический Тогда
            Значения.Добавить(Объект.Родитель);
        КонецЕсли;
        Если МетаданныеОбъекта.Владельцы.Количество() > 0 Тогда
            Значения.Добавить(Объект.Владелец);
        КонецЕсли;
```

```
    КонецЕсли;
    Для Каждого ТабличнаяЧасть Из МетаданныеОбъекта.ТабличныеЧасти Цикл
        Для Каждого СтрокаТЧ Из Объект[ТабличнаяЧасть.Имя] Цикл
            Для Каждого Реквизит Из ТабличнаяЧасть.Реквизиты Цикл
                Значения.Добавить(СтрокаТЧ[Реквизит.Имя]);
            КонецЦикла;
        КонецЦикла;
    КонецЦикла;

    ТипыСправочников = Справочники.ТипВсеСсылки();
    Для Каждого Значение Из Значения Цикл
        Если Не ЗначениеЗаполнено(Значение) Или Не
ТипыСправочников.СодержитТип(ТипЗнч(Значение)) Тогда
            Продолжить;
        КонецЕсли;
        Если Добавленные[Значение] <> Неопределено Тогда
            Продолжить;
        КонецЕсли;
        Добавленные.Вставить(Значение, Истина);
        Если НеПередаватьСправочник(Значение.Метаданные().Имя) Тогда
            Продолжить;
        КонецЕсли;
        ОбъектЗависимости = Значение.ПолучитьОбъект();
        Если ОбъектЗависимости <> Неопределено Тогда
            Зависимости.Добавить(ОбъектЗависимости);
        КонецЕсли;
    КонецЦикла;

КонецПроцедуры

// Служебные справочники, которые не передаются (пользователи, файлы,
идентификаторы метаданных).
Функция НеПередаватьСправочник(Имя)

    Возврат
СтрНайти(",Пользователи,ВнешниеПользователи,ГруппыПользователей,Идентификатор
ыОбъектовМетаданных,"
        + "ИдентификаторыОбъектовРасширений,", "," + Имя + ",") > 0
        Или СтрНайти(Имя, "ПрисоединенныеФайлы") > 0;

КонецФункции

#КонецОбласти

#Область Загрузка

// Загружает все входящие файлы канала в порядке их создания.
// Обработанные файлы переносятся в processed, ошибочные - в error.
Процедура Загрузить(Роль, Канал, Протокол)

    КаталогВходящих = КаталогОбмена() + "\" + Канал + "\incoming";
    Файлы = НайтиФайлы(КаталогВходящих, "*.json", Ложь);
    Если Файлы.Количество() = 0 Тогда
        Протокол.Добавить(ВремяСобытия() + " Канал " + Канал + ": входящих
сообщений нет");
        Возврат;
    КонецЕсли;

    Список = Новый СписокЗначений;
    Для Каждого Файл Из Файлы Цикл
        Список.Добавить(Файл.ПолноеИмя, Файл.Имя);
    КонецЦикла;
    Список.СортироватьПоПредставлению();
```

```
    ИмяПолученных = Роль + "_" + Канал + "_received";
    Полученные = ПрочитатьСостояние(ИмяПолученных);
    Если Полученные = Неопределено Тогда
        Полученные = Новый Соответствие;
    КонецЕсли;
    ИмяСоответствий = Роль + "_mapping";
    Соответствия = ПрочитатьСостояние(ИмяСоответствий);
    Если Соответствия = Неопределено Тогда
        Соответствия = Новый Соответствие;
    КонецЕсли;

    Для Каждого ЭлементСписка Из Список Цикл

        ПолноеИмяФайла = ЭлементСписка.Значение;
        ИмяФайла = ЭлементСписка.Представление;

        Попытка

            Пакет = ПрочитатьПакет(ПолноеИмяФайла);
            Если Пакет.Канал <> Канал Тогда
                ВызватьИсключение "Сообщение относится к каналу " +
Пакет.Канал + ", ожидался " + Канал;
            КонецЕсли;

            Если Полученные[Пакет.ИдентификаторСообщения] <> Неопределено
Тогда
                ПереместитьВАрхив(ПолноеИмяФайла, Канал, "processed");
                ЗаписатьВЖурнал(Роль, Протокол, "Канал " + Канал + ":
сообщение " + ИмяФайла
                    + " уже было загружено ранее, повторно не
обрабатывается");
                Продолжить;
            КонецЕсли;

            Итог = ЗагрузитьПакет(Пакет, Соответствия);

            Полученные.Вставить(Пакет.ИдентификаторСообщения,
XMLСтрока(ТекущаяДатаСеанса()));
            ЗаписатьСостояние(ИмяПолученных, Полученные);
            ЗаписатьСостояние(ИмяСоответствий, Соответствия);
            ПереместитьВАрхив(ПолноеИмяФайла, Канал, "processed");
            ЗаписатьВЖурнал(Роль, Протокол, "Канал " + Канал + ": загружено
сообщение " + ИмяФайла + ". " + Итог);

        Исключение

            ТекстОшибки = КраткоеПредставлениеОшибки(ИнформацияОбОшибке());
            Попытка
                ПереместитьВАрхив(ПолноеИмяФайла, Канал, "error");
            Исключение
                ТекстОшибки = ТекстОшибки + " (файл не перемещен в error)";
            КонецПопытки;
            ЗаписатьВЖурнал(Роль, Протокол, "Канал " + Канал + ": сообщение "
+ ИмяФайла + " не загружено: "
                + ТекстОшибки, Истина);

        КонецПопытки;

    КонецЦикла;

КонецПроцедуры

Функция ПрочитатьПакет(ПолноеИмяФайла)
```

```
    Чтение = Новый ЧтениеJSON;
    Чтение.ОткрытьФайл(ПолноеИмяФайла, "UTF-8");
    Попытка
        Пакет = СериализаторXDTO.ПрочитатьJSON(Чтение);
    Исключение
        Чтение.Закрыть();
        ВызватьИсключение;
    КонецПопытки;
    Чтение.Закрыть();

    Если ТипЗнч(Пакет) <> Тип("Структура")
        Или Не Пакет.Свойство("Объекты")
        Или Не Пакет.Свойство("Зависимости")
        Или Не Пакет.Свойство("ИдентификаторСообщения") Тогда
        ВызватьИсключение "Неверный формат сообщения обмена";
    КонецЕсли;

    Возврат Пакет;

КонецФункции

// Записывает объекты сообщения. Возвращает краткий итог.
Функция ЗагрузитьПакет(Пакет, Соответствия)

    Замены = Новый Соответствие; // ссылка из сообщения -> ссылка в этой базе
    Создано = 0;
    Обновлено = 0;
    Сопоставлено = 0;
    Подробности = Новый Массив;

    // 1. Связанные элементы справочников: существующие не изменяются,
недостающие
    // сопоставляются по правилам или создаются с тем же уникальным
идентификатором.
    Для Каждого Объект Из Пакет.Зависимости Цикл

        Ссылка = Объект.Ссылка;
        Локальная = ЛокальнаяСсылка(Ссылка, Соответствия);
        Если Локальная <> Неопределено Тогда
            Если Локальная <> Ссылка Тогда
                Замены.Вставить(Ссылка, Локальная);
            КонецЕсли;
            Продолжить;
        КонецЕсли;

        Найденная = НайтиПоПравилам(Объект, Замены);
        Если Найденная <> Неопределено Тогда
            Замены.Вставить(Ссылка, Найденная);
            ЗапомнитьСоответствие(Соответствия, Ссылка, Найденная);
            Сопоставлено = Сопоставлено + 1;
            Подробности.Добавить("сопоставлен: " + Найденная);
            Продолжить;
        КонецЕсли;

        ЗаменитьСсылки(Объект, Замены);
        Объект.ОбменДанными.Загрузка = Истина;
        Объект.Записать();
        Создано = Создано + 1;
        Подробности.Добавить("создан связанный элемент: " + Объект.Ссылка);

    КонецЦикла;

    // 2. Основные объекты сообщения: отсутствующие создаются, существующие
обновляются.
```

```
    Для Каждого Объект Из Пакет.Объекты Цикл

        Ссылка = Объект.Ссылка;
        ЭтоДокумент = ОбщегоНазначения.ЭтоДокумент(Объект.Метаданные());
        Локальная = ЛокальнаяСсылка(Ссылка, Соответствия);

        Если Локальная = Неопределено И Не ЭтоДокумент Тогда
            Найденная = НайтиПоПравилам(Объект, Замены);
            Если Найденная <> Неопределено Тогда
                ЗапомнитьСоответствие(Соответствия, Ссылка, Найденная);
                Сопоставлено = Сопоставлено + 1;
                Подробности.Добавить("сопоставлен с существующим: " +
Найденная);
                Продолжить;
            КонецЕсли;
        ИначеЕсли Локальная <> Неопределено И Локальная <> Ссылка Тогда
            Подробности.Добавить("уже сопоставлен ранее: " + Локальная);
            Продолжить;
        КонецЕсли;

        Существует = (Локальная <> Неопределено);
        ЗаменитьСсылки(Объект, Замены);

        Если ЭтоДокумент Тогда
            Состояние = ЗаписатьДокумент(Объект, Существует);
        Иначе
            Объект.ОбменДанными.Загрузка = Истина;
            Объект.Записать();
            Состояние = "";
        КонецЕсли;

        Если Существует Тогда
            Обновлено = Обновлено + 1;
            Подробности.Добавить("обновлен: " + Объект.Ссылка + Состояние);
        Иначе
            Создано = Создано + 1;
            Подробности.Добавить("создан: " + Объект.Ссылка + Состояние);
        КонецЕсли;

    КонецЦикла;

    Возврат "Создано: " + Создано + ", обновлено: " + Обновлено + ",
сопоставлено: " + Сопоставлено
        + ?(Подробности.Количество() > 0, " (" + СтрСоединить(Подробности, ";
") + ")", "");

КонецФункции

// Документ записывается в режиме загрузки, затем, если он был проведен в
источнике,
// проводится в этой базе обычным образом (формируются движения).
Функция ЗаписатьДокумент(Объект, Существует)

    Ссылка = Объект.Ссылка;
    Если Существует Тогда
        ОтменитьПроведение(Ссылка);
    КонецЕсли;

    ПроведенВИсточнике = Объект.Проведен;
    Объект.ОбменДанными.Загрузка = Истина;
    Объект.Записать();

    Если Не ПроведенВИсточнике Тогда
        Возврат ", не проведен";
```

```
    КонецЕсли;

    Попытка
        ДокументОбъект = Ссылка.ПолучитьОбъект();
        ДокументОбъект.Записать(РежимЗаписиДокумента.Проведение);
        Возврат ", проведен";
    Исключение
        ТекстОшибки = КраткоеПредставлениеОшибки(ИнформацияОбОшибке());
        ОтменитьПроведение(Ссылка);
        Возврат ", записан без проведения: " + ТекстОшибки;
    КонецПопытки;

КонецФункции

Процедура ОтменитьПроведение(Ссылка)

    Попытка
        ДокументОбъект = Ссылка.ПолучитьОбъект();
        Если ДокументОбъект <> Неопределено И ДокументОбъект.Проведен Тогда
            ДокументОбъект.Записать(РежимЗаписиДокумента.ОтменаПроведения);
        КонецЕсли;
    Исключение
        // Документ останется в прежнем состоянии; ошибка проведения уже
попадет в протокол
    КонецПопытки;

КонецПроцедуры

// Ссылка в этой базе для ссылки из сообщения: по таблице соответствий или по
тому же
// уникальному идентификатору. Неопределено, если объекта в этой базе нет.
Функция ЛокальнаяСсылка(Ссылка, Соответствия)

    Идентификатор = Соответствия[КлючСсылки(Ссылка)];
    Если Идентификатор <> Неопределено Тогда
        Менеджер = ОбщегоНазначения.МенеджерОбъектаПоСсылке(Ссылка);
        Локальная = Менеджер.ПолучитьСсылку(Новый
УникальныйИдентификатор(Идентификатор));
        Если ОбщегоНазначения.ССылкаСуществует(Локальная) Тогда
            Возврат Локальная;
        КонецЕсли;
    КонецЕсли;

    Если ОбщегоНазначения.ССылкаСуществует(Ссылка) Тогда
        Возврат Ссылка;
    КонецЕсли;

    Возврат Неопределено;

КонецФункции

// Правила сопоставления: Контрагенты - по ИНН и КПП, остальные справочники -
по наименованию
// (подчиненные - в пределах владельца). Сопоставление выполняется только при
единственном совпадении.
Функция НайтиПоПравилам(Объект, Замены)

    МетаданныеОбъекта = Объект.Метаданные();
    Если Не ОбщегоНазначения.ЭтоСправочник(МетаданныеОбъекта) Или
МетаданныеОбъекта.ДлинаНаименования = 0 Тогда
        Возврат Неопределено;
    КонецЕсли;

    Запрос = Новый Запрос;
```

```
    Условия = Новый Массив;
    Условия.Добавить("НЕ Т.ПометкаУдаления");

    Если МетаданныеОбъекта.Имя = "Контрагенты" И
ЗначениеЗаполнено(Объект.ИНН) Тогда
        Условия.Добавить("Т.ИНН = &ИНН");
        Условия.Добавить("Т.КПП = &КПП");
        Запрос.УстановитьПараметр("ИНН", Объект.ИНН);
        Запрос.УстановитьПараметр("КПП", Объект.КПП);
    Иначе
        Условия.Добавить("Т.Наименование = &Наименование");
        Запрос.УстановитьПараметр("Наименование", Объект.Наименование);
    КонецЕсли;

    Если МетаданныеОбъекта.Иерархический
        И МетаданныеОбъекта.ВидИерархии =
Метаданные.СвойстваОбъектов.ВидИерархии.ИерархияГруппИЭлементов Тогда
        Условия.Добавить("Т.ЭтоГруппа = &ЭтоГруппа");
        Запрос.УстановитьПараметр("ЭтоГруппа", Объект.ЭтоГруппа);
    КонецЕсли;
    Если МетаданныеОбъекта.Владельцы.Количество() > 0 Тогда
        Владелец = Замены[Объект.Владелец];
        Условия.Добавить("Т.Владелец = &Владелец");
        Запрос.УстановитьПараметр("Владелец", ?(Владелец = Неопределено,
Объект.Владелец, Владелец));
    КонецЕсли;

    Запрос.Текст = "ВЫБРАТЬ ПЕРВЫЕ 2 Т.Ссылка КАК Ссылка ИЗ " +
МетаданныеОбъекта.ПолноеИмя()
        + " КАК Т ГДЕ " + СтрСоединить(Условия, " И ");
    Выборка = Запрос.Выполнить().Выбрать();
    Если Выборка.Количество() <> 1 Тогда
        Возврат Неопределено;
    КонецЕсли;
    Выборка.Следующий();
    Возврат Выборка.Ссылка;

КонецФункции

Процедура ЗапомнитьСоответствие(Соответствия, ВнешняяСсылка, ЛокальнаяСсылка)

    Соответствия.Вставить(КлючСсылки(ВнешняяСсылка),
Строка(ЛокальнаяСсылка.УникальныйИдентификатор()));

КонецПроцедуры

// Заменяет в объекте ссылки из сообщения на сопоставленные ссылки этой базы.
Процедура ЗаменитьСсылки(Объект, Замены)

    Если Замены.Количество() = 0 Тогда
        Возврат;
    КонецЕсли;

    МетаданныеОбъекта = Объект.Метаданные();
    Имена = Новый Массив;
    Для Каждого Реквизит Из МетаданныеОбъекта.Реквизиты Цикл
        Имена.Добавить(Реквизит.Имя);
    КонецЦикла;
    Если ОбщегоНазначения.ЭтоСправочник(МетаданныеОбъекта) Тогда
        Если МетаданныеОбъекта.Иерархический Тогда
            Имена.Добавить("Родитель");
        КонецЕсли;
        Если МетаданныеОбъекта.Владельцы.Количество() > 0 Тогда
            Имена.Добавить("Владелец");
```

```
            КонецЕсли;
        КонецЕсли;
        Для Каждого Имя Из Имена Цикл
            НовоеЗначение = Замены[Объект[Имя]];
            Если НовоеЗначение <> Неопределено Тогда
                Объект[Имя] = НовоеЗначение;
            КонецЕсли;
        КонецЦикла;

        Для Каждого ТабличнаяЧасть Из МетаданныеОбъекта.ТабличныеЧасти Цикл
            Для Каждого СтрокаТЧ Из Объект[ТабличнаяЧасть.Имя] Цикл
                Для Каждого Реквизит Из ТабличнаяЧасть.Реквизиты Цикл
                    НовоеЗначение = Замены[СтрокаТЧ[Реквизит.Имя]];
                    Если НовоеЗначение <> Неопределено Тогда
                        СтрокаТЧ[Реквизит.Имя] = НовоеЗначение;
                    КонецЕсли;
                КонецЦикла;
            КонецЦикла;
        КонецЦикла;

КонецПроцедуры

#КонецОбласти

#Область ФайлыИЖурнал

Процедура ПодготовитьКаталоги()

    Каталоги = Новый Массив;
    Для Каждого Канал Из СтрРазделить("branch_to_head,head_to_branch", ",")
Цикл
        Для Каждого Подкаталог Из СтрРазделить("incoming,processed,error",
",") Цикл
            Каталоги.Добавить(КаталогОбмена() + "\" + Канал + "\" +
Подкаталог);
        КонецЦикла;
    КонецЦикла;
    Каталоги.Добавить(КаталогОбмена() + "\state");
    Каталоги.Добавить(КаталогОбмена() + "\log");

    Для Каждого ИмяКаталога Из Каталоги Цикл
        Каталог = Новый Файл(ИмяКаталога);
        Если Не Каталог.Существует() Тогда
            СоздатьКаталог(ИмяКаталога);
        КонецЕсли;
    КонецЦикла;

КонецПроцедуры

Процедура ПереместитьВАрхив(ПолноеИмяФайла, Канал, Подкаталог)

    Файл = Новый Файл(ПолноеИмяФайла);
    ИмяВАрхиве = КаталогОбмена() + "\" + Канал + "\" + Подкаталог + "\" +
Файл.Имя;
    ФайлВАрхиве = Новый Файл(ИмяВАрхиве);
    Если ФайлВАрхиве.Существует() Тогда
        УдалитьФайлы(ИмяВАрхиве);
    КонецЕсли;
    ПереместитьФайл(ПолноеИмяФайла, ИмяВАрхиве);

КонецПроцедуры

Функция ПрочитатьСостояние(Имя)
```

```
    ИмяФайла = КаталогОбмена() + "\state\" + Имя + ".json";
    Файл = Новый Файл(ИмяФайла);
    Если Не Файл.Существует() Тогда
        Возврат Неопределено;
    КонецЕсли;

    Чтение = Новый ЧтениеJSON;
    Чтение.ОткрытьФайл(ИмяФайла, "UTF-8");
    Данные = ПрочитатьJSON(Чтение, Истина);
    Чтение.Закрыть();
    Возврат Данные;

КонецФункции

Процедура ЗаписатьСостояние(Имя, Данные)

    ИмяФайла = КаталогОбмена() + "\state\" + Имя + ".json";
    ВременныйФайл = ИмяФайла + ".tmp";

    Запись = Новый ЗаписьJSON;
    Запись.ОткрытьФайл(ВременныйФайл, "UTF-8", Ложь, Новый
ПараметрыЗаписиJSON(ПереносСтрокJSON.Авто, Символы.Таб));
    ЗаписатьJSON(Запись, Данные);
    Запись.Закрыть();

    Файл = Новый Файл(ИмяФайла);
    Если Файл.Существует() Тогда
        УдалитьФайлы(ИмяФайла);
    КонецЕсли;
    ПереместитьФайл(ВременныйФайл, ИмяФайла);

КонецПроцедуры

// Пишет событие в общий журнал C:\UNFExchange\log\exchange.log, в журнал
регистрации и в протокол.
Процедура ЗаписатьВЖурнал(Роль, Протокол, Текст, ЭтоОшибка = Ложь)

    СтрокаЖурнала = ВремяСобытия() + " [" + Роль + "] " + ?(ЭтоОшибка,
"ОШИБКА: ", "") + Текст;
    Протокол.Добавить(СтрокаЖурнала);

    Попытка
        Запись = Новый ЗаписьТекста(КаталогОбмена() + "\log\exchange.log",
КодировкаТекста.UTF8, , Истина);
        Запись.ЗаписатьСтроку(СтрокаЖурнала);
        Запись.Закрыть();
    Исключение
        // Каталог обмена недоступен - событие останется в журнале
регистрации
    КонецПопытки;

    ЗаписьЖурналаРегистрации("Обмен Филиал - Головной офис",
        ?(ЭтоОшибка, УровеньЖурналаРегистрации.Ошибка,
УровеньЖурналаРегистрации.Информация), , , СтрокаЖурнала);

КонецПроцедуры

Функция ВремяСобытия()

    Возврат Формат(ТекущаяДатаСеанса(), "ДФ='dd.ММ.уууу ЧЧ:mm:ss'");

КонецФункции

Функция КлючСсылки(Ссылка)
```

```
    Возврат Ссылка.Метаданные().ПолноеИмя() + "|" +
Строка(Ссылка.УникальныйИдентификатор());

КонецФункции

#КонецОбласти

#КонецЕсли
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Модуль формы обработки «ИмпортРасходныхНакладных»

```
#Область ОбработчикиСобытийФормы

&НаСервере
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

    ПутьJSON = "C:\Exchange\incoming.json";

КонецПроцедуры

#КонецОбласти

#Область ОбработчикиСобытийЭлементовФормы

&НаКлиенте
Процедура ПутьJSONНачалоВыбора(Элемент, ДанныеВыбора, СтандартнаяОбработка)

    СтандартнаяОбработка = Ложь;
    ВыбратьФайл(Неопределено);

КонецПроцедуры

#КонецОбласти

#Область ОбработчикиКомандФормы

// Кнопка "Загрузить JSON": выбор файла и загрузка его содержимого в
обработку.
&НаКлиенте
Процедура Загрузить(Команда)

    ВыбратьФайл(Новый ОписаниеОповещения("ЗагрузитьВыбранныйФайл",
ЭтотОбъект));

КонецПроцедуры

// Кнопка "Создать накладные": создание расходных накладных по загруженному
файлу.
&НаКлиенте
Процедура Создать(Команда)

    Если ПустаяСтрока(ТекстJSON) Тогда
        Сообщить("Загрузите JSON-файл кнопкой ""Загрузить JSON"" (после
создания документов файл загружается заново).");
        Возврат;
    КонецЕсли;

    СоздатьДокументыНаСервере();

КонецПроцедуры

#КонецОбласти

#Область СлужебныеПроцедурыИФункции

&НаКлиенте
Процедура ВыбратьФайл(ОповещениеПослеВыбора)
```

```
    Диалог = Новый ДиалогВыбораФайла(РежимДиалогаВыбораФайла.Открытие);
    Диалог.Заголовок = "Выберите JSON-файл с накладными";
    Диалог.Фильтр = "Файлы JSON (*.json)|*.json|Все файлы (*.*)|*.*";
    Если Не ПустаяСтрока(ПутьJSON) Тогда
        ТекущийФайл = Новый Файл(ПутьJSON);
        Диалог.Каталог = ТекущийФайл.Путь;
        Диалог.ПолноеИмяФайла = ТекущийФайл.Имя;
    КонецЕсли;
    Диалог.Показать(Новый ОписаниеОповещения("ВыборФайлаЗавершение",
ЭтотОбъект, ОповещениеПослеВыбора));

КонецПроцедуры

&НаКлиенте
Процедура ВыборФайлаЗавершение(ВыбранныеФайлы, ОповещениеПослеВыбора) Экспорт

    Если ВыбранныеФайлы = Неопределено Или ВыбранныеФайлы.Количество() = 0
Тогда
        Возврат;
    КонецЕсли;

    ПутьJSON = ВыбранныеФайлы[0];
    Если ОповещениеПослеВыбора <> Неопределено Тогда
        ВыполнитьОбработкуОповещения(ОповещениеПослеВыбора);
    КонецЕсли;

КонецПроцедуры

&НаКлиенте
Процедура ЗагрузитьВыбранныйФайл(Результат, ДополнительныеПараметры) Экспорт

    Попытка
        ЗагрузитьНаСервере();
    Исключение
        Сообщить("Файл не загружен: " +
КраткоеПредставлениеОшибки(ИнформацияОбОшибке()));
        Возврат;
    КонецПопытки;
    Сообщить("Документов в файле: " + Строка(ЧислоЗаписей));

КонецПроцедуры

// Читает файл, проверяет структуру JSON и показывает накладные на форме.
&НаСервере
Процедура ЗагрузитьНаСервере()

    ТекстJSON = "";
    ЧислоЗаписей = 0;
    СписокНакладных.Очистить();
    ТекстСообщения = "";

    Чтение = Новый ЧтениеТекста(ПутьJSON, КодировкаТекста.UTF8);
    Текст = Чтение.Прочитать();
    Чтение.Закрыть();

    Накладные = ПрочитатьНакладныеИзJSON(Текст);

    ТекстJSON = Текст;
    ЧислоЗаписей = Накладные.Количество();
    Для Каждого ИсходныйДок Из Накладные Цикл
        НоваяСтрока = СписокНакладных.Добавить();
        НоваяСтрока.Дата = XMLЗначение(Тип("Дата"),
ИсходныйДок.ДатаДокумента);
```

```
        НоваяСтрока.Организация = ИсходныйДок.Организация;
        НоваяСтрока.Контрагент = ИсходныйДок.Контрагент;
        НоваяСтрока.Склад = ИсходныйДок.Склад;
        НоваяСтрока.Позиций = ИсходныйДок.Товары.Количество();
        Для Каждого Позиция Из ИсходныйДок.Товары Цикл
            НоваяСтрока.Сумма = НоваяСтрока.Сумма + Позиция.Сумма;
        КонецЦикла;
        НоваяСтрока.Результат = "загружена";
    КонецЦикла;
    ТекстСообщения = "Загружен файл " + ПутьJSON + ". Документов в файле: " +
ЧислоЗаписей + ".";

КонецПроцедуры

// Разбирает текст JSON и проверяет его структуру.
//
// Возвращаемое значение:
//  Массив из Структура - накладные с ключами ДатаДокумента, Организация,
Контрагент, Склад, Товары.
//
&НаСервереБезКонтекста
Функция ПрочитатьНакладныеИзJSON(Текст)

    Чтение = Новый ЧтениеJSON;
    Чтение.УстановитьСтроку(Текст);
    Данные = ПрочитатьJSON(Чтение);
    Чтение.Закрыть();

    Если ТипЗнч(Данные) <> Тип("Массив") Тогда
        ВызватьИсключение "Корень JSON должен быть массивом.";
    КонецЕсли;

    ОбязательныеКлючи =
СтрРазделить("ДатаДокумента,Организация,Контрагент,Склад,Товары", ",");
    КлючиСтроки = СтрРазделить("Номенклатура,Количество,Сумма", ",");

    Для Каждого ОбъектJSON Из Данные Цикл

        Если ТипЗнч(ОбъектJSON) <> Тип("Структура") Тогда
            ВызватьИсключение "Элемент массива - не объект.";
        КонецЕсли;

        Для Каждого Ключ Из ОбязательныеКлючи Цикл
            Если Не ОбъектJSON.Свойство(Ключ) Тогда
                ВызватьИсключение "В накладной нет обязательного ключа """ +
Ключ + """.";
            КонецЕсли;
        КонецЦикла;

        ТоварыJSON = Неопределено;
        Если Не ОбъектJSON.Свойство("Товары", ТоварыJSON)
            Или ТипЗнч(ТоварыJSON) <> Тип("Массив") Тогда
            ВызватьИсключение "Нет массива Товары.";
        КонецЕсли;

        Для Каждого Позиция Из ТоварыJSON Цикл
            Для Каждого Ключ Из КлючиСтроки Цикл
                Если ТипЗнч(Позиция) <> Тип("Структура") Или Не
Позиция.Свойство(Ключ) Тогда
                    ВызватьИсключение "В строке товаров нет ключа """ + Ключ
+ """.";
                КонецЕсли;
            КонецЦикла;
        КонецЦикла;
```

```
    КонецЦикла;

    Возврат Данные;

КонецФункции

// Создает расходные накладные. Сначала проверяются все ссылки по всем
документам;
// если хотя бы одна не найдена или неоднозначна, не создается ни один
документ.
&НаСервере
Процедура СоздатьДокументыНаСервере()

    Накладные = ПрочитатьНакладныеИзJSON(ТекстJSON);

    // 1. Сопоставление по наименованиям и предварительная проверка
    Найденные = Новый Соответствие;
    Ошибки = Новый Массив;
    Для Каждого ИсходныйДок Из Накладные Цикл
        НайтиПоНаименованию("Организации", ИсходныйДок.Организация,
Найденные, Ошибки);
        НайтиПоНаименованию("Контрагенты", ИсходныйДок.Контрагент, Найденные,
Ошибки);
        НайтиПоНаименованию("СтруктурныеЕдиницы", ИсходныйДок.Склад,
Найденные, Ошибки);
        Для Каждого Позиция Из ИсходныйДок.Товары Цикл
            НайтиПоНаименованию("Номенклатура", Позиция.Номенклатура,
Найденные, Ошибки);
        КонецЦикла;
    КонецЦикла;

    Если Ошибки.Количество() > 0 Тогда
        Для Каждого ТекстОшибки Из Ошибки Цикл
            Сообщить(ТекстОшибки);
        КонецЦикла;
        ТекстСообщения = "Документы не созданы. Ошибки сопоставления:" +
Символы.ПС + СтрСоединить(Ошибки, Символы.ПС);
        Возврат;
    КонецЕсли;

    // 2. Создание документов в одной транзакции
    Протокол = Новый Массив;
    НачатьТранзакцию();
    Попытка
        Для Индекс = 0 По Накладные.ВГраница() Цикл
            Док = НоваяРасходнаяНакладная(Накладные[Индекс], Найденные);
            Док.Записать(РежимЗаписиДокумента.Запись); // без проведения
            ТекстРезультата = "создана № " + Док.Номер + " от " +
Формат(Док.Дата, "ДЛФ=D") + ", не проведена";
            СписокНакладных[Индекс].Результат = ТекстРезультата;
            Протокол.Добавить("Расходная накладная № " + Док.Номер + " от " +
Формат(Док.Дата, "ДЛФ=D")
                + ": " + Док.Контрагент + ", строк " +
Док.Запасы.Количество()
                + ", сумма " + Формат(Док.СуммаДокумента, "ЧДЦ=2; ЧН=0") + "
(записана, не проведена)");
        КонецЦикла;
        ЗафиксироватьТранзакцию();
    Исключение
        ОтменитьТранзакцию();
        ТекстОшибки = "Документы не созданы: " +
КраткоеПредставлениеОшибки(ИнформацияОбОшибке());
        Сообщить(ТекстОшибки);
```

```
        ТекстСообщения = ТекстОшибки;
        Возврат;
    КонецПопытки;

    // Защита от повторного создания тех же документов: для нового создания
файл нужно загрузить заново
    ТекстJSON = "";
    ТекстСообщения = "Создано расходных накладных: " + Накладные.Количество()
+ Символы.ПС + СтрСоединить(Протокол, Символы.ПС);
    Сообщить("Создано расходных накладных: " + Накладные.Количество());

КонецПроцедуры

// Строгий поиск элемента справочника по наименованию: ошибка, если элемент
// не найден или найдено несколько элементов с таким наименованием.
&НаСервереБезКонтекста
Процедура НайтиПоНаименованию(ИмяСправочника, Наименование, Найденные,
Ошибки)

    Ключ = ИмяСправочника + "|" + Наименование;
    Если Найденные[Ключ] <> Неопределено Тогда
        Возврат;
    КонецЕсли;

    МетаданныеСправочника = Метаданные.Справочники[ИмяСправочника];
    Если ПустаяСтрока(Наименование) Тогда
        ТекстОшибки = "Не заполнено наименование (справочник """ +
МетаданныеСправочника.Синоним + """).";
        Если Ошибки.Найти(ТекстОшибки) = Неопределено Тогда
            Ошибки.Добавить(ТекстОшибки);
        КонецЕсли;
        Возврат;
    КонецЕсли;

    ТекстЗапроса =
    "ВЫБРАТЬ ПЕРВЫЕ 2
    |    Т.Ссылка КАК Ссылка
    |ИЗ
    |    Справочник." + ИмяСправочника + " КАК Т
    |ГДЕ
    |    Т.Наименование = &Наименование
    |    И НЕ Т.ПометкаУдаления";
    Если МетаданныеСправочника.Иерархический
        И МетаданныеСправочника.ВидИерархии =
Метаданные.СвойстваОбъектов.ВидИерархии.ИерархияГруппИЭлементов Тогда
        ТекстЗапроса = ТекстЗапроса + "
        |    И НЕ Т.ЭтоГруппа";
    КонецЕсли;

    Запрос = Новый Запрос(ТекстЗапроса);
    Запрос.УстановитьПараметр("Наименование", Наименование);
    Выборка = Запрос.Выполнить().Выбрать();

    Если Не Выборка.Следующий() Тогда
        Ошибки.Добавить("Не найден элемент справочника """ +
МетаданныеСправочника.Синоним + """: " + Наименование);
        Возврат;
    КонецЕсли;
    Ссылка = Выборка.Ссылка;
    Если Выборка.Следующий() Тогда
        Ошибки.Добавить("Несколько совпадений в справочнике """ +
МетаданныеСправочника.Синоним + """: " + Наименование);
        Возврат;
    КонецЕсли;
```

```
    Найденные.Вставить(Ключ, Ссылка);

КонецПроцедуры

// Заполняет новую расходную накладную по объекту JSON. Номер не заполняется:
// его присвоит платформа (автонумерация) при записи.
&НаСервереБезКонтекста
Функция НоваяРасходнаяНакладная(ИсходныйДок, Найденные)

    Док = Документы.РасходнаяНакладная.СоздатьДокумент();
    Док.Дата = XMLЗначение(Тип("Дата"), ИсходныйДок.ДатаДокумента);

    ДанныеЗаполнения = Новый Структура;
    ДанныеЗаполнения.Вставить("ВидОперации",
Перечисления.ВидыОпераций РасходнаяНакладная.ПродажаПокупателю);
    ДанныеЗаполнения.Вставить("Организация", Найденные["Организации|" +
ИсходныйДок.Организация]);
    ДанныеЗаполнения.Вставить("Контрагент", Найденные["Контрагенты|" +
ИсходныйДок.Контрагент]);
    ДанныеЗаполнения.Вставить("СтруктурнаяЕдиница",
Найденные["СтруктурныеЕдиницы|" + ИсходныйДок.Склад]);

    // Заполнение по умолчанию средствами УНФ (валюта, курс, налогообложение,
ответственный и т.д.)
    Док.Заполнить(ДанныеЗаполнения);
    ЗаполнитьЗначенияСвойств(Док, ДанныеЗаполнения);
    Док.Дата = XMLЗначение(Тип("Дата"), ИсходныйДок.ДатаДокумента);

    Если Не ЗначениеЗаполнено(Док.Договор) Тогда
        Док.Договор =
Справочники.ДоговорыКонтрагентов.ПоДокументуКонтрагентуОрганизацииИВидуОперац
ии(
            Док, Док.Контрагент, Док.Организация, Док.ВидОперации);
    КонецЕсли;
    Если Не ЗначениеЗаполнено(Док.Договор) Тогда
        Док.Договор =
Справочники.ДоговорыКонтрагентов.ДоговорПоУмолчанию(Док.Контрагент);
    КонецЕсли;

    ВидСтавкиОрганизации =
ОбщегоНазначения.ЗначениеРеквизитаОбъекта(Док.Организация,
"ВидСтавкиНДСПоУмолчанию");
    Для Каждого Позиция Из ИсходныйДок.Товары Цикл

        Номенклатура = Найденные["Номенклатура|" + Позиция.Номенклатура];

        СтрокаТЧ = Док.Запасы.Добавить();
        СтрокаТЧ.Номенклатура = Номенклатура;
        СтрокаТЧ.ЕдиницаИзмерения = Номенклатура.ЕдиницаИзмерения;
        СтрокаТЧ.Количество = Позиция.Количество;
        СтрокаТЧ.Сумма = Позиция.Сумма;
        СтрокаТЧ.Цена = ?(СтрокаТЧ.Количество = 0, 0, Окр(СтрокаТЧ.Сумма /
СтрокаТЧ.Количество, 2));

        Если Док.НалогообложениеНДС =
Перечисления.ТипыНалогообложенияНДС.ОблагаетсяНДС Тогда
            ВидСтавки = ?(ЗначениеЗаполнено(Номенклатура.ВидСтавкиНДС),
Номенклатура.ВидСтавкиНДС, ВидСтавкиОрганизации);
            СтрокаТЧ.СтавкаНДС = Справочники.СтавкиНДС.СтавкаНДС(ВидСтавки,
Док.Дата);
            Ставка =
УправлениеНебольшойФирмойПовтИсп.ПолучитьЗначениеСтавкиНДС(СтрокаТЧ.СтавкаНДС
);
```

```
        Иначе
            СтрокаТЧ.СтавкаНДС =
УправлениеНебольшойФирмойПовтИсп.ПолучитьСтавкуНДСБезНДС();
            Ставка = 0;
        КонецЕсли;

        СтрокаТЧ.СуммаНДС = ?(Док.СуммаВключаетНДС,
            СтрокаТЧ.Сумма - СтрокаТЧ.Сумма / ((Ставка + 100) / 100),
            СтрокаТЧ.Сумма * Ставка / 100);
        СтрокаТЧ.Всего = СтрокаТЧ.Сумма + ?(Док.СуммаВключаетНДС, 0,
СтрокаТЧ.СуммаНДС);

    КонецЦикла;

    Док.СуммаДокумента = Док.Запасы.Итог("Всего");

    Возврат Док;

КонецФункции

#КонецОбласти
```